

SIEMENS EDA

Generating Libraries for Catapult

Software Version 2025.2_1
June 2025

SIEMENS

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this Documentation shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: www.plm.automation.siemens.com/global/en/legal/trademarks.html. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center: support.sw.siemens.com

Send Feedback on Documentation: support.sw.siemens.com/doc_feedback_form

Table of Contents

Chapter 1: Creating Memory Libraries	2
1.1. Generating Memory Libs using GUI	3
1.1.1. <i>Select Technology Files</i>	4
1.1.2. <i>Memory and HDL/Library Parameters</i>	6
1.1.3. <i>Specifying the Model Ports</i>	8
1.1.4. <i>Generating the Model</i>	10
1.2. Creating Memory Libraries from a file	11
Chapter 2: Characterizing ASIC Base Libraries	20
2.1. Preparing to Create a Library	20
2.1.1. <i>Basic RTL Technology Settings</i>	21
2.1.2. <i>Specifying Wire Loads</i>	21
2.1.3. <i>Determining Technology Libraries for RTL Synthesis</i>	21
2.1.4. <i>Determining Licensing Approach</i>	22
2.2. Creating the Library	23
<i>Setting the Working Directory</i>	24
<i>Command line library creation</i>	24
<i>Creating a New Library manually</i>	25
2.3. Creating a New Library with the Library Characterization Wizard	26
<i>Technology Specifications</i>	26
<i>Settings (1): Catapult GUI and Library Modules</i>	28
<i>Settings (2): RTL Synthesis Libraries and Library Module Usage</i>	29
<i>Settings (3): Optional Files/Settings for other RTL Synthesis flows</i>	30
2.4. Characterizing Libraries or Components	31
2.4.1. <i>Spot Checking Library Settings</i>	32
2.4.2. <i>Running Characterization</i>	32
2.4.3. <i>Determining Characterization Points</i>	35
2.4.4. <i>Automatic smoothing of ASIC Library Data</i>	37
2.4.5. <i>Queuing Multiple Libraries for Multi-day Run</i>	38
2.4.6. <i>Determining the Fastest Clock Period using Iterative Estimation</i>	38
2.5. Evaluating Library Characterization Results	39
2.5.1. <i>Delay Characterization Properties</i>	40
2.5.2. <i>Resetting the Data Before Another Characterization</i>	41
2.5.3. <i>Exporting the Characterization Datapath</i>	41
2.5.4. <i>Plotting the Characterization Data</i>	42
2.5.5. <i>Adding/Removing QMODs</i>	44

Table of Contents

2.5.6. Viewing the Characterization Transcript.....	46
2.5.7. Reducing Characterization Time.....	47
2.6. Saving Libraries.....	48
2.7. Troubleshooting Library Failures.....	49
2.8. Improving the Accuracy of Estimated Characterization Values.....	52
2.8.1. Specifying Characterization Curve Shape.....	52
2.8.2. Editing Characterization Properties.....	55
2.9. Using the Library Farm.....	56
2.9.1. Setting Up Library Farm Hosts.....	57
Configuring Library Farm to Use the Load Sharing Facility (LSF) software.....	58
Configuring Library Farm to Use the Sun Grid Engine (SGE) software.....	60
Chapter 3: Editing Libraries.....	62
3.1. Updating Catapult Libraries with New Components.....	62
3.2. Editing Library Variables.....	64
3.3. Editing Module Parameters.....	75
3.4. Editing Module Ports.....	76
3.5. Editing Module Bindings.....	78
3.6. Editing Module Pin Associations.....	80
3.7. Editing Module Property Mappings.....	81
Chapter 4: Third-Party Information.....	83

Chapter 1: Creating Memory Libraries

Memory Generator within Catapult allows you to import VHDL, Verilog, or Liberty memory models and generate a memory library that can be used within Catapult. This tool will also generate a SystemC model of the memory and SystemC transactors that can be used with SCVerify. It also adds a dependency to the imported HDL and/or Liberty file for the netlisted RTL design. If you specify a datasheet (.ds) or PDF file, Catapult will add a link to display this file from the Architecture Constraints window.

During library compilation, Memory Generator writes all files to a temp directory within your specified Output directory. After it verifies that the compilation was successful, Memory Generator moves the files into your specified output directory and removes the temp directory. If there was an error, the files in the temp directory are intended to be used for debugging purposes.

The Memory Generator is accessible from the Task Bar, the Libraries Constraint Editor and the Tools menu. It provides a Wizard-like GUI that guides you through the process. Each of the screens in the wizard has online help. The Memory Generator GUI also performs dynamic error checks so you can isolate any memory issues prior to generating or using the memory within Catapult.

The Memory Generator can handle the following types of memory models:

- Single or Multiple Port memories (eg. Dual-port RAMs). Each Port must have it's own address, clock and data lines.
- Handle Read/Write conflict resolution
- Address and Data bus widths can be hard-coded or using generics (VHDL) or parameters(verilog)
- Reset can be defined as async/sync or active high/low.
- Chip-wide or Byte-wise write enables
- Register Files as memories
- Memories with latencies greater than 1.
- Pins on the memories can be automatically tied high/low.

The following list describes the open issues with using the Memory Generator:

- Dualport support. The control signals for all ports must have the same phase (eg. Active high/Active low).
- Async memory read. Memories with 1 readwrite port where the write port is synchronous and the read port is asynchronous are not supported. Async memory read ports can not have a Chip Select.
- Direct pin types are not connected correctly for SystemC designs. New ports are added to the top level interface. Expect ports to be connected by name for inferred SystemC memories.

- Verilog localparms are not supported.
- Include files in the memory must have absolute pathnames. If the paths are relative, then you must either concatenate the files together or use absolute paths.
- Latency / Delay values apply to all ports. Memories having ports with different timings are not supported.

1.1. Generating Memory Libs using GUI

The Memory Generator allows you to read existing HDL simulation memory models into Catapult and store them as libraries for re-use in other projects. You can access the Memory Generator from the Task bar, Tools menu or Libraries constraint editor. After you open the Memory Generator, follow the steps in the wizard. Whenever the Memory Generator requires a field or detects an incorrect value, it will turn the field to red.

The following sections describe each form within the Memory Generator:

1. Select Technology Files
2. Memory and HDL/Library Parameters
3. Specifying the Model Ports
4. Generating the Model

This section uses the following Verilog RAM model to generate the screen shots of the Memory Generator:

```

1 module RAMexample (Q, CLK, CSN, WEN, A, D);
2     parameter words = 'd16;
3     parameter width = 'd16;
4     parameter addr_width = 4;
5
6     input [width-1:0]      D;    // Data input
7     input [addr_width-1:0] A;    // Read/write address
8     input                 CSN;  // Active low chip select
9     output reg [width-1:0] Q;   // Data output
10    input                 CLK;
11    input                 WEN;  // Active low write enable
12
13    reg [width-1:0] mem [words-1:0];
14    // synopsys translate_off
15    // <RAM functionality>
16    // synopsys translate_on
17 endmodule

```

1.1.1. Select Technology Files

In the first form of the Memory Generator, you can specify one or more files associated with a single memory model. The resulting memory library will be saved in the Catapult working directory. When you have completed this form, you can click the **Import** button and Catapult will parse the specified files and attempt to pre-populate the Parameter and Port information.

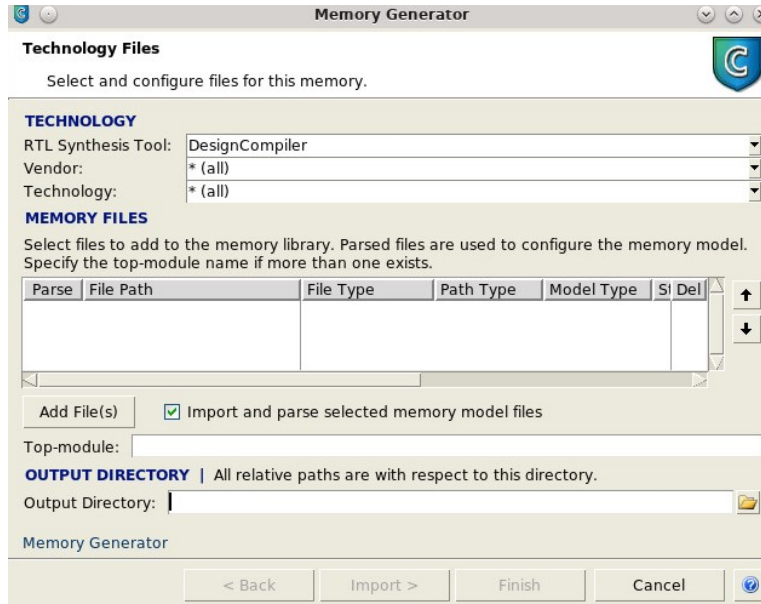


Illustration 1: Technology Files pane in Memory Generator

Catapult should be able to parse the name, direction, and width of the pins in the model from VHDL and Verilog files.

Base Technology Library

The following three fields denote the default base library that will be used with the memory model. Memory Generator creates a listbox for each field based on the information that is currently available to Catapult.

- **RTL Synthesis Tool** specifies the flow that was used to characterize the base library. Each library is specific to the synthesis tool. If you use multiple synthesis tools, then you will need multiple libraries. Memory Generator pre-populates this list based on the Search Path defined in the Libraries Constraints Editor. The Catapult libraries have a pre-determined set of supported flows (synthesis tools).
- **Vendor** specifies the ASIC or FPGA vendor of the library. This listbox is pre-populated with the Vendor names of all libraries that are found in the library search path. You can select a specific Vendor or use the '*' character to have Catapult make the memory available for all vendors.
- **Technology** specifies the process point of the library (eg. 65 nm). This listbox is pre-populated with the Technology names for the Vendors specified in the Vendor Listbox. You can select a specific Technology or use the '*' character to have Catapult make the memory available for all technologies.

Import Memory Files

The main part of this form allows you to add one or more files associated with a single ram into the Memory Generator. During the file import, Catapult will parse the files and attempt to pre-populate the remaining forms in the Memory Generator. This section has the following options:

Using the **Add Files** button, you can add multiple files and use the **Parse** column checkbox to determine whether Memory Generator should attempt to extract values from the model file.

- Catapult determines the language of the memory model for the **File Type** column based on the file extension. Even if the model is not parsed, it will still be added to the memory library as a file dependency. Verilog models must use *.v and VHDL models must use *.vhd or *.vhd. Liberty files use a .lib extension and the Datasheet can use either .ds or .pdf.
- The **Path Type** column controls the final pathname and location of any memory files (eg. Verilog).
 - *Copy Local* stores the dependant files in the *Output Directory* specified at the bottom of this form.
 - *Relative* stores the dependant file pathnames relative to the *Output Directory* path. No dependant files are copied.
 - *Absolute* stores the dependant file pathnames as an absolute path from it's current location. No dependant files are copied.
- The **Model Type** column specifies which downstream should use the dependant file. The generic setting will include the file in the dependency list for all simulation and synthesis flows.
- The **Static Flag** determines whether Catapult should *copy* or *point* to the referenced file when a solution contains this memory. If set to *yes*, the file path is included in the file dependencies. If set to *no*, the file is copied into the solution when the netlister [PackageOutput](#) option is true.
- **VHDL Library Mapping:** specifies logical and physical library mapping that may be required to use the VHDL model of the Memory. If your VHDL memory model references another library outside of 'work', then you must supply the library mapping in this field.

Import Library Data from memory model allows you to override an existing settings in the Memory Generator with the currently specified HDL file. If this checkbox is selected, Memory Generator will parse the names of the pins and parameters/generics in the memory model HDL file and pre-populate the corresponding forms. Unchecking this option allows you to make changes to other options on this form without overriding any previously entered data on other forms within the Memory Generator.

Top Module denotes the name of the top level memory model. Some models may contain multiple modules/entities. This value should be the name of the top level Verilog module or VHDL entity.

Output Directory specifies the location to store the generated memory. This directory will contain the generated Catapult library for the memory (<module>.lib), SystemC models for simulation and a (optionally) simple test design to verify the memory in SCVerify. This field must contain a pathname. If an absolute pathname is not used, the memory library is stored relative to the current Working Directory specified in Catapult.

1.1.2. Memory and HDL/Library Parameters

In this form, you can configure the parameters on the imported memory model. The Memory Generator attempts to read the parameters from the imported VHDL and Verilog memory files. If any invalid data is specified in the form, Memory Generator will change the background of the field to light red. The following illustration shows the completed pane for the example verilog model at the beginning of this section. The highlighted areas denote updated entries.

Memory and Module Parameters

Configure this memory's parameters.

MEMORY PARAMETERS

Field	Value
Module name	RAMexample
Library	RAMexample
Data Width	width
Number of Words	words
Area	0
Read / Write Behavior	Undefined
Read Latency	1

Fields above can utilize parameter variables below in place of explicit values.

HDL & LIBRARY PARAMETERS

Ignore	Type	Parameter	Default	Min	Max	Del
☐	HDL	words	16			X
☐	HDL	width	16			X
☐	HDL	addr_width	4			X

Add Library Parameter

Memory Generator

< Back Next > Finish Cancel

Illustration 2: Module Parameters pane in Memory Generator

This pre-determined list of parameters allow Catapult to map the functionality of the HDL model to the internal functions of Catapult's memory operators. Each of these fields will accept either numbers or HDL/library parameter names. You can not use expressions.

- **Module Name** contains the module/entity name from the HDL memory file.
- **Library** contains the name of the library.
- **Data Width** specifies the number of bits on the data bus. This value includes the entire width of the data bus regardless of whether the data bus uses write/read masks. This value may be a hard-coded number or a module parameter (eg. data_width).

NOTE: It is recommended that you always specify the parameter name from the HDL model in this entry. If parameter in the HDL model uses a hard-coded value that will never change, you should specify the hard-coded value as the **default** value for the parameter in the **HDL & Libraries parameters** section at the bottom of the form pane.

- **Number of Words** specifies the number of bits in the word. Catapult uses this value for Bitwise Write Enables.

NOTE: *It is recommended that you always specify the parameter name from the HDL model in this entry. If parameter in the HDL model uses a hard-coded value that will never change, you should specify the hard-coded value as the **default** value for the parameter in the **HDL & Libraries parameters** section at the bottom of the form pane.*

- **Area** specifies the size of the memory in library units. Each base technology library may use a different unitless measure to specify the relative size of all cells in the library. Catapult adds this value to all area summary and BOM reports to give a more accurate measure of the overall area usage of the design.
- **Read / Write Behavior** specifies the Read/Write resolution—whether the Read or Write function should have precedence when the design attempts to read and write to the same memory address. This information should be designed into the HDL memory model logic. For example, if you specify Read before Write for Read/Write Behavior, then when the design attempts to read and write at the same memory address, the memory will act as follows:

1. read the current value of the memory address to the data output
2. write the data input value to the memory address

If you select "Unknown", then the HDL memory model was designed to produce random values on the data output.

- **Read Latency** specifies the number of clock cycles that Catapult should schedule for each read operation. This value must be either an integer or a parameter name on the HDL model.
- **Write Latency** specifies the number of clock cycles that Catapult should schedule for each write operation. This value must be either an integer or a parameter name on the HDL model.
- **Read Delay (ns)** specifies the propagation delay from the active edge of the read clock until the data is available on the output pin of the memory model.
- **Write Delay (ns)** specifies the time for asynchronous writes. If your memory does not have this functionality, leave it as the default value.
- **Setup Time (ns)** specifies the time that the input data must remain stable prior to the active write clock edge.
- **Init Delay** specifies the number of clock cycles before starting the next accesses to this port. The read or write latency can be greater than this value. For example, if this value is set to '2', then the memory can be pipelined down to II=2, but not II=1.
- **VHDL Array Path** specifies the signal name of the internal 2-D array that stores the memory values within the HDL model. This value is only used in the SLEC flow.
- **Verilog Array Path** specifies the signal name of the internal 2-D array that stores the memory values within the HDL model. This value is only used in the SLEC flow.

HDL & Library Parameters

This section defines the parameters that can be used in other sections of the memory library in place of

explicit values. There are two types of parameters: 1) *HDL parameters*, those imported from the HDL memory model itself, and 2) *user-defined library parameters*. You can specify default, minimum, and maximum values for each parameter. Where allowed, leaving the values blank will automatically use predetermined default values.

All HDL parameters must be utilized somewhere within the memory generator configuration or explicitly set to *Ignore* in the parameter configuration. All user-defined library parameters must have *Min* and *Max* defined to bound the parameter.

It is recommended that you mark all parameters as Ignore except the data width, address width and number of words. For these three parameters, you should set the default values to match the parameter values in the simulation model. This setting will ensure that the parameters have the correct value when simulating the RAM.

After you complete this step, click the **Next>** button to proceed to specifying the model ports.

1.1.3. Specifying the Model Ports

The *Port and Pin Configuration* form in the Memory Generator allows you to specify single-port, dual-port or multi-port memory models. These memory models may contain memories or arrays of registers. The following illustration denotes the completed form pane for the example verilog model at the beginning of the section:

Memory Generator

Port and Pin Configuration
Create and configure ports. Map and configure pins.

MEMORY PORTS | Select a predefined port configuration or add/modify memory ports directly.

Single RW port (1rw)

Port	Port Identifier	Port Mode	Del
rw	port_0	ReadWrite	X

Add Port

PINS | Assign pin functions, then map pins to ports. Click port-headers to toggle checkboxes.

Pin	Direction	Width	Pin Function	Phase	Default	rw
Q	out	width	Data Output	N/A		☒
CLK	in	1.0	Clock	Active High		☒
CSN	in	1.0	Chip Select	Active Low		☒
WEN	in	1.0	Write Enable	Active Low		☒
A	in	addr_width	Address	N/A		☒
D	in	width	Data Input	N/A		☒

Memory Generator

< Back Next > Finish Cancel

Illustration 3: Module Parameters pane in Memory Generator

Within the Memory Generator, there are difference between a *memory port* and the *port defined in the VHDL/Verilog model* (which will be referred to as the Physical Pin (Pin)). The Memory generator uses the following definitions for these terms:

- A **Port** defines the number of address buses that can access the Catapult memory model. Each port configuration defines how the set of physical pins on the model will be connected. During optimization, Catapult will search the memories in the library for available memory operations. A memory may have

one or more ports for each mode (eg. single-port read, dual-port write, single-port readwrite).

- A **Pin** is the pin defined in the VHDL entity or Verilog module. A Pin can be defined on multiple ports in the Catapult memory model.

For each Memory Port, you will need to define the following information:

- **Port** displays a color-coded, generated id that is used within the Memory Generator. It is a read-only field. Each of this values are added to the Pin mapping table at the bottom of the form.
- **Port Identifier** allows you to specify the port name that will be used within the transactor of the SystemC model.
- **Port Mode** allows you to specify the type of port.
- **Del** allows you to delete a port from the tables.

For each pin, you must map functionality of the pin to a pre-defined list of functions that are available for the port. You will also need to assign each pin to one (or more) Memory Ports. Memory Generator checks for invalid entries in each row. The following list describes the columns in the PINS table:

- **Pin** infers the list of pins from the imported VHDL/Verilog source model.
- **Direction** infers the direction of the pin from the imported VHDL/Verilog source model.
- **Width** infers the number of bits on the Physical Pin defined in the previous Parameters form.
- **Pin Function** defines the functionality of the Physical Pin. You can view descriptions of the available pin functions by hovering your mouse over the Pin Function listbox of the respective pin. These pin functions use common terms for memory pins with the following exceptions that are specific to Catapult (neither of these options apply to SystemC designs):
 - **Direct** connects the Pin from the memory to the top level interface. Catapult will add additional ports as needed to route the pin through hierarchy. Each direct pin will have a unique port on the top level interface. Catapult uses the Pin name on the memory model as the top-level port name.
 - **Global** connects all Pins with the same name to a single port on the top level interface. Catapult uses the Pin name on the memory model as the top-level port name.
- **Phase** specifies the relevant phase options for the pin.
- **Default** specifies the initial value on the pin. This value should only be used to set a pin on the memory model to a constant.
- **Port Assignment** is a series of checkboxes that allow you to group a Pin with a Port. Memory Generator will automatically stipple-out any ports that are not relevant to a specific pin. For example, a Read Enable pin would not be used with a Write Port.

NOTE: *The Direct and Global functions do not apply to SystemC designs.*

NOTE: If you need to drive a pin with a constant value, set the Pin Function to Unconnected with a Default value of '1' or '0'.

NOTE: You should mark all non-memory-access related pins (eg. Bist pins) as **Direct** and set their default value to configure the RAM for normal operation.

NOTE: All "Unconnected" input pins must be set to a default value to ensure quality design practices. "Unconnected" output pins may be left dangling. If a pin is not to be used by Catapult, but still has some design function, you may want to declare the input pin as either "direct" or "global". This declaration would allow you to connect it at the top level of the design.

After you complete this step, click the **Next>** button to proceed to generating the model ports.

1.1.4. Generating the Model

In this pane of the Memory Generator, you will generate the library model, SystemC simulation model, SystemC transactor, and a TCL script for the memory. Catapult executes the TCL script in Catapult to generate the memory files. You can re-purpose this TCL file to generate other similar memories. Any errors will be reported in the transcript of the Memory Generator.

In the Output Folder specified in the initial dialog, Catapult will write a compiled library model (*.lib), SystemC simulation and transaction models for verification (*.h) for the requested memory-based models. The basename for all of the generated models is the Module Name.

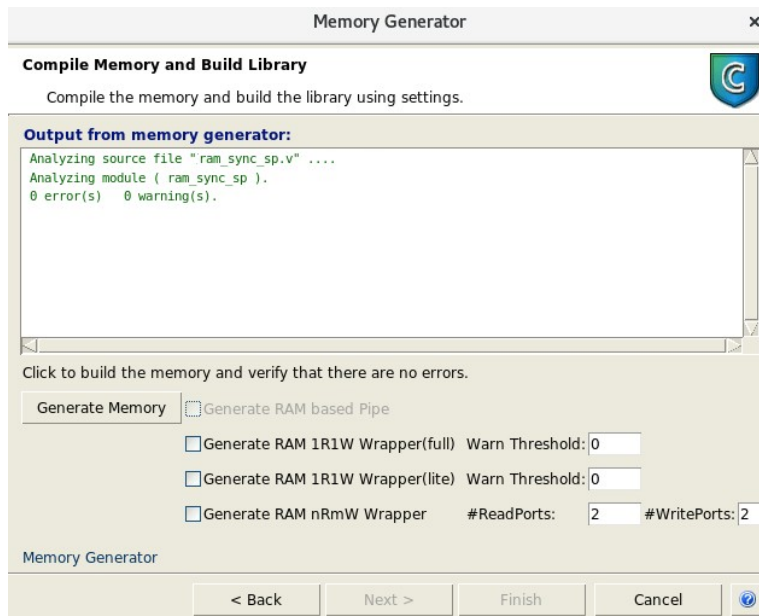


Illustration 4: Generating the Memory Library

If you specified the one of the "Generate RAM based Pipe" checkbox, then Catapult will also generate a memory-based pipe component in addition to the memory within the same Catapult library. For an example of generating and using a custom memory-based pipe in Catapult, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Libraries > Generating Memory-Based Pipes**).

If your source memory is a Single-Port RAM, Memory Generator provides the ability to generate wrappers

around your single port RAM that will emulate simple dual port RAMs (1R1W and nRmW). If those checkboxes are selected, MemGen will create the corresponding SystemVerilog wrapper file and a MemGenTcl file in the Output directory and then compile the new memory into a Catapult library.

After you have successfully generated the memory library, you can make the memory accessible to Catapult by adding the directory containing the Catapult memory library to one of the following locations:

- *Component Library Search Path Option.* In Catapult, select the Component Libraries pane in the Options form (Tools > Set Options) and add the directory path to the list.
- *Library Constraint Editor:* In Catapult, select the Libraries task and click on the Search Path button to add the directory containing the Catapult memory library.

After Catapult loads the memory library, it will be an available *Resource Type* to use when specifying Architecture constraints (eg. Mapping arrays in the core design to the memory).

1.2. Creating Memory Libraries from a file

In addition to using the [Memory Generator GUI](#), you can also specify the functionality and parameters for memory libraries using the TCL file. This flow can be very useful in the situations where you need to generate libraries of several similar memories.

The Memory Generator creates a TCL file containing the necessary parameters for generating a Catapult memory library. This section includes the reference information for the fields within the TCL file.

To generate a memory from the file, you must load the Memory Generator flow and then run the MemoryGenerator_BuildLib option. Because there are a significant number of fields in the memory generator specification, it is highly recommended that you generate the initial memory using the Memory Generator GUI. The GUI will produce a template in the library output directory that you can copy and modify for subsequent memories. The basic format of the Memory Generator file is :

```
flow package require MemGen
flow run /MemGen/MemoryGenerator_BuildLib {
  <memory file fields>
}
```

NOTE: All values are required and must exist in the TCL file otherwise the library will be generated with empty values and may not function as expected. This file adheres to TCL syntax rules. All values in the file are case sensitive.

Since the file is read into Catapult via the command line, all fields within the memory specification file must adhere to TCL syntax rules. The following table describes the main fields of Memory TCL file. The remaining tables describe the syntax of the FILES, PARAMETERS, PORTS and PINMAPS sections of the Memory TCL file.

Table 1: MemoryGenerator_BuildLib file format

Keyword	Values	Default	Description
AREA	integer	0	Size of the memory in default area units of the

			base technology library.
COMP_PKG	String	none	Specify the VHDL component package name. This value is stored in the NETLIST section of the Module within the Catapult library. If this field is not used, the Memory/FIFO generator will issue a warning about possible VHDL netlisting issues when the Output/PackageOutput option is set to <i>true</i> and the STATICFILES entry in the FILES section is <i>false</i> .
DEPTH	Integer	0	Number of addressable locations within the memory. The default value of "0" implies that the full address width is available to access the memory locations.
FILES	file_struct		List of file dependencies for the memory. Catapult includes this path name in the dependency file when the RTL netlist is written. This list can include one or more files. Refer to Table 2, for information on the file_struct syntax.
GEN_RAM_PIPE	boolean	false	Flag to inform the Memory Generator to create an additional library component for a memory-based pipe (in addition to the RAM model).
GEN_EXT_RAM_PIPE	boolean	false	Flag to inform the Memory Generator to create an additional library component for a pipe with an external memory (in addition to the RAM model).
INITDELAY	Integer	1	Initialization cycles. This value defines how many clock cycles the memory needs to write a value into memory.
INPUTDELAY	Float	0.01	Propagation Delay on the input pins.
LIBRARY	String	Module name from HDL file	Base Library name corresponding to the memory library. Catapult evaluates the Library Search Path to locate memory libraries that are associated with a Base technology library. When you select a base library in the GUI, Catapult will also display all supplemental library that are associated with the base library. If you specify a library name of "*", this memory library will be associated with all base libraries.
MODULE	String	Module name from HDL file	Memory name. This value should correspond to the module name in the verilog file or entity name in the VHDL file. Catapult uses this name when it instantiates the memory in the netlist.
OUTPUT_DIR	Pathname		Pathname to write the resulting library files. On successful generation of the memory, this directory will contain the .lib memory library for catapult and the SystemC models for simulation.

			This path can contain an absolute or relative pathname.
PARAMETERS	param_list	Empty list	List of memory parameters. In the GUI flow, this list is pre-populated when the HDL simulation model is imported. <i>NOTE: The values in this structure are not used to generate the library. This structure may be used in a future release.</i>
PINMAPS	pinmap_list	none	Maps pins on the simulation model to pins on Catapult's internal memory port components. Refer to for more information on the items in the PINMAP Section.
PORTS	port_list	none	List of Port Definitions. This list defines the names of the available port modes for the memory (eg. Read, Write). Refer to for more information on the items in the PORTS section.
RAM_RWRAP	Integer	2	Specifies the number of Read Ports to implement when the RAM_WRAPPER2 option is set to true. This option is only valid for Single Port Memories (with RAM_WRAPPER2=true).
RAM_THOLD	Integer	2	This value is used when the RAM_WRAPPER0 option is enabled.
RAM_THOLDL	Integer	2	This value is used when the RAM_WRAPPER1 option is enabled.
RAM_WRAPPER	boolean	false	Deprecated. This option has been replaced with RAM_WRAPPER0
RAM_WRAPPER0	boolean	false	Flag to inform Memory Generator to create a HDL wrapper and new memory module that emulates a 1R1W memory from a Single Port memory. This component enables you to use a SPRAM as a Shared Memory. This flag is only valid with Single Port Memories.
RAM_WRAPPER1	boolean	false	Flag to inform Memory Generator to create a HDL wrapper and new memory module that emulates a 1R1W memory (lite) from a Single Port memory. This component enables you to use a SPRAM as a Shared Memory. This flag is only valid with Single Port Memories.
RAM_WRAPPER2	boolean	false	Flag to inform Memory Generator to create a HDL wrapper and new memory module that emulates a N-Read, M-Write memory from a Single Port memory. This component enables you to use a SPRAM as a Shared Memory. This flag is only valid with Single Port Memories. Specify the number of Read/Write ports using RAM_RWRAP and RAM_WWRAP.

RAM_WWRAP	Integer	2	Specifies the number of Write Ports to implement when the RAM_WRAPPER2 option is set to true. This option is only valid for Single Port Memories (with RAM_WRAPPER2=true).
RDWRRESOLUTION	RBW WBR UNKNOWN	UNKNOWN	Defines how the memory handles reading and writing to the same address on the same clock cycle. RBW (0): Reads the values from memory prior to writing a new value WBR (2): Writes the new value to memory and propagates the new value to the output UNKNOWN (1): Write X(s) to the memory and output MemGen maps the corresponding numeric value (denoted in parenthesis) to the <i>RdWrResolution</i> property on the <i>all</i> binding of the module.
READDELAY	Float	0.1	Clock-to-Q delay of the memory during a read operation
READLATENCY	Integer or HDL parameter	1	Number of clock cycles to complete the read operation
RTLTOOL	flow name	DesignCompiler	Name of the flow that will be used for RTL synthesis of the output netlist containing the memory. This tool will be used with the 'go synthesize' command. The value can only contain one tool name. Wildcards are not allowed. This value must match the RTL tool name in the base library. The templates within Library Builder use a fixed number of flow names so these values should be consistent across multiple libraries. Catapult uses this value as the initial criteria for displaying libraries in the Libraries Constraint Editor.
TECHNOLOGY	String *	none	Name of the Technology used with the Base Library. This value must match the base library name or the wildcard character. You can not use partial wildcards (eg. 'tsmc*'). The wildcard character implies that it is valid for all technologies in the specified flow.
TIMEUNIT	String	1ns	Specifies the timeunit of the delay fields for this memory. The value must be: <1 10 100><ps ns>
VENDOR	String *	none	Name of the Vendor used with the Base Library. This value must match the base library name or the wildcard character. You can not use partial wildcards (eg. '1*'). This wildcard character implies that the memory library is valid for all

			vendors in the specified flow.
VERILOGARRAYPATH	array_name	{}	Specifies the name of the array within the simulation model that contains the 2-D memory values. This value is only used by the SLEC verification tool.
VHDLARRAYPATH	array_name	{}	Specifies the name of the array within the simulation model that contains the 2-D memory values. This value is only used by the SLEC verification tool.
VHDL_LIB_MAPS	<mapping>	{work ./work}	One or more items that map the logical library name that are referenced in the VHDL model to the physical directory. By default, all simulation models are mapped to the work library.
WIDTH	Integer or parameter	{}	Width of the data port. This value can be a parameter or integer value.
WRITEDELAY	float	0.1	Propagation delay prior to writing a value to a memory with an asynchronous write. This value must be a positive, non-zero value. If you do not have an asynch write, use the default value.
WRITELATENCY	Integer or HDL parameter	1	Number of clock cycles to complete a MEMORYWRITE operation.

FILES Section

Table 2 describes the format for the list item included within the FILES section. This section contains one list item for each HDL file that is required by the simulation memory model. These files will be included in the memory library as file dependencies. Whenever this memory is included in an output netlist, these simulation files will be referenced in the dependency file for the netlist. The following text shows the format of the FILES section:

```
FILES {
    {FILENAME <file_path> FILETYPE <file_type> MODELTYPE <modeltype>}
    . . .
    {FILENAME <file_path> FILETYPE <file_type> MODELTYPE <modeltype>}
}
```

Here is an example of the FILES section for a memory model with a reference to a single HDL simulation model:

```
FILES {
    {FILENAME /u/catuser/mem/ram.v FILETYPE Verilog MODELTYPE GENERIC}
}
```

Table 2: Syntax of FILES Section

Object	Description
FILENAME <i>file_path</i>	Pathname to the simulation model. This value can be absolute or relative. Relative pathnames are relative to the Catapult working directory when you run the Memory Generator
FILETYPE <i>file_type</i>	HDL language of the reference simulation model. This value can be either VHDL or VERILOG.
MODELTYPE <i>model_type</i>	This value should be set to "GENERIC". At this time, the MODELTYPE is not used during library compilation. This field will be used in a future enhancement to the Memory Generator. It is included in this release for future compatibility.
STATICFILE <i>flag_value</i>	This flag specifies whether Catapult should <i>copy</i> or <i>point</i> to the referenced file when a solution contains this memory. If set to <i>1</i> , the file path is included in the file dependencies. If set to <i>0</i> , the file is copied into the solution when the netlister PackageOutput option is true.

PARAMETERS Section

Table 3 describes the format for the list item included within the PARAMETERS section. This section contains one list item for each parameter from the simulation memory model that Catapult would need to modify to implement the variations in the memory. When using the Memory Generator GUI, each parameter in the HDL simulation model is added during import.

The following text shows the format of the PARAMETERS section:

```
PARAMETERS {
    {PARAMETER <name> MIN <min_value> MAX <max_value> DEFAULT <default_value>}
    . . .
    {PARAMETER <name> MIN <min_value> MAX <max_value> DEFAULT <default_value>}
}
```

Table 3: Syntax of the PARAMETERS Section

Object	Description
PARAMETER <name>	Name of the parameter used in the HDL simulation model.
MIN <min_value>	Minimum value that Catapult can use for utilizing this memory. This value can be an integer or float value. If there is no minimum value, then specify an empty list ("{}") for the <min_value>.
MAX <max_value>	Maximum value that Catapult can use for utilizing this memory. This value can be an integer or float value. If there is no minimum value, then specify an empty list ("{}") for the <max_value>.
DEFAULT <default_value>	Default value for this parameter. This value can be a float or integer value. If

	there is no minimum value, then specify an empty list ("{}") for the <default_value>.
--	---

PINMAP Section

Table 4 describes the format for the list item included within the PINMAPS section. This section contains one list item for each pin on the simulation memory model that Catapult needs to connect when it instantiates the memory in a netlist. The following text shows the format of the PINMAPS section:

```
PINMAPS {
    {PHYPIN <phy_name> LOGPIN <log_name> <FUNCTION> DIRECTION <dir> WIDTH
<width> PHASE<phase> DEFAULT <default_value> PORTS <port_list> }
    . . .
    {PHYPIN <phy_name> LOGPIN <log_name> <FUNCTION> DIRECTION <dir> WIDTH
<width> PHASE<phase> DEFAULT <default_value> PORTS <port_list> }
}
```

Here is an example of the PINMAPS section for a memory model with a reference to a single HDL simulation model:

```
PINMAPS {
    {PHYPIN clk LOGPIN CLOCK DIRECTION in WIDTH 1.0 PHASE 1 \
        DEFAULT {} PORTS {r w}}
    {PHYPIN reb LOGPIN READ_ENABLE DIRECTION in WIDTH 1.0 PHASE 1 \
        DEFAULT {} PORTS r}
    {PHYPIN dout LOGPIN DATA_OUT DIRECTION out WIDTH dwidth PHASE {} \
        DEFAULT {} PORTS r}
    {PHYPIN addr LOGPIN ADDRESS DIRECTION in WIDTH awidth PHASE {} \
        DEFAULT {} PORTS [r w]}
    {PHYPIN web LOGPIN WRITE_ENABLE DIRECTION in WIDTH 1.0 PHASE 1 \
        DEFAULT {} PORTS w}
    {PHYPIN din LOGPIN DATA_IN DIRECTION in WIDTH data_width PHASE {} \
        DEFAULT {} PORTS w}
}
```

Table 4: Syntax of PINMAPS Section

Object	Description
PHYPIN <phy_name>	Name of the pin in the HDL simulation model.
LOGPIN <pin_function>	Name of the pin on the internal Catapult memory operator. Valid values are: ADDRESS: address for any type of port DATA_IN: Data to be written into the memory

	• DATA_OUT: Data read out of the memory • WRITE_ENABLE: Single bit write control • READ_ENABLE: single bit read control • PORT_ENABLE: single bit enable for the port which is asserted during either read or write operations • WRITE_MASK: Write control with more than 1 bit. Supports partial word operations • READ_MASK: Controls reads of less than a full word • CHIP_SELECT: Enable/disable ALL memory operations (but pipelines continue to flush) • DIRECT: Pin that is routed to the top level interface. All direct pins will have unique names on the top level. • GLOBAL: Pin that is routed to top level interface. All global pins with the same name will be connected. • UNCONNECTED: Output pins will not be connected. Input pins will be tied to the value of the DEFAULT field. • CLOCK: Clock for the memory • ENABLE: Clock enable. This pin 'freezes' the pipelines in the memory if de-asserted. • A_RST: Asynchronous reset (of the memory pipeline) • S_RST: Synchronous reset (of the memory pipeline)
DIRECTION <dir>	Direction of the data flow for the pin. Valid values are 'in', 'out', 'inout'.
WIDTH <width>	Number of bits of the PHYPIN. Valid values are either an integer or a string with the name of a parameter that is defined in the PARAMETERS section.
PHASE <phase>	Active level or edge of the phypin. Valid values are '1' for active high or rising edge, '0' for active low or falling edge, or {} if the phypin is not sensitive to the level or edge of the signal (eg. Address and data buses).
DEFAULT <default_value>	Initial value of phypin. Valid values are '1' to initialize high, '0' to initialize low, '{}' to not initialize, or a string with the name of a parameter. At this time, this field is only used to define constants on UNCONNECTED phypins.
PORTS <port_list>	One or more PORT names that are defined in the PORTS section

PORTS Section

Table 5 describes the format for the list item included within the PORTS section. This section contains one list item for each port of the memory model. Each pin in the PINMAP section can use the PORTS object in the PINMAP section to map to a port defined in this PORTS section. The following text shows the format of the PINMAPS section:

```
PORTS {
    {PORT <name> MODE <type> }
    . . .
    {PORT <name> MODE <type> }
}
```

Here is an example of the PINMAPS section for a memory model with a reference to a single HDL simulation model:

```
PORTS {
    {PORT r MODE Read }
    {PORT w MODE Write }
}
```

Table 5: Syntax of the PORTS Section

Object	Description
PORT <name>	Defines the name of the PORT. This value will be referenced in the PORT list in the PINMAP section.
MODE <mode>	Name of the memory mode on the memory operator. The values of mode are case-sensitive. Valid values are: - Read - Write - ReadWrite - AsyncRead

Chapter 2: Characterizing ASIC Base Libraries

Catapult requires libraries created and characterized with Library Builder. For FPGA technologies, Catapult ships pre-characterized libraries for all supported FPGA devices. For ASIC technologies, you must create and characterize a custom ASIC library (unless a Catapult library for your technology is provided by the foundry).

This chapter describes the procedure and best practices for creating and characterizing a new ASIC library using the Base ASIC Library template provided with Library Builder.

1. [Preparing to Create a Library](#)
2. [Creating the Library](#)
3. [Characterizing Libraries or Components](#)
4. [Evaluating Library Characterization Results](#)
5. [Saving Libraries](#)

NOTE: If this library will only be used within Catapult Ultra using the on-the-fly methodology, then you do not need to perform the above-mentioned characterization or evaluation steps because Catapult will characterize any necessary operators as needed by the optimizer or scheduler.

The following sections describe each of these steps in detail:

- [Troubleshooting Library Failures](#)
- [Improving the Accuracy of Estimated Characterization Values](#)
- [Using the Library Farm](#)

2.1. Preparing to Create a Library

A custom ASIC library is created from the *Base ASIC Library* template and the technology library used with your RTL synthesis tool of choice. Library Builder generates the RTL components, runs them through RTL synthesis to gather timing, area, etc data, and saves that data to an ASIC library for Catapult.

Currently Library Builder supports characterization with Synopsys Design Compiler and Mentor Graphics' Oasys-RTL.

To ensure that the library characterization is adequate, you should prepare for library as described in the following topics:

- [Basic RTL Technology Settings](#)

- [Specifying Wire Loads](#)
- [Determining Technology Libraries for RTL Synthesis](#)
- [Determining Licensing Approach](#)

2.1.1. Basic RTL Technology Settings

The settings used for characterizing the ASIC library must match those used in your *in-house* RTL flow. Catapult generates RTL code that will likely be combined with other RTL code and synthesized into ASIC gates. If the synthesis flow used to characterize the new ASIC library differs from your standard flow, the area and timing estimates in Catapult are likely to be inaccurate.

You may need to contact your silicon vendor or your RTL synthesis team to determine what settings to use for the following RTL synthesis options:

- **Operating conditions** — defines the process, temperature, and voltage settings.
- **Driving cell** — specifies the technology cell driving the inputs of the characterized operator. We recommend setting the driving cell value to the smallest register in the library. Although this option is interchangeable with the *Input transition* setting but specifying the driving cell is recommended.
- **External load** — specifies the capacitive load attached to the outputs of the operator during characterization. We recommend setting the loading capacitive value to 10X of the input “D” pin on the smallest register in your library. By default, the Library Wizard should extract this 10X value from the liberty files. Increasing this load would make library more pessimistic.
- **Input transition** — specifies the input transition value. If the *Driving cell* option is used, you should leave this variable blank.
- **Other advanced settings** — extra settings such as boundary optimization, extra optimization settings, and so on. These options are based on individual preference, license availability, foundry/company guidelines etc.

2.1.2. Specifying Wire Loads

It is common for technology libraries to contain a default wire load. However oftentimes a wire load is responsible for poor correlation between Catapult and RTL synthesis estimations therefore we recommend against using wire loads.

To ensure that the wire loads are not used set the *interconnect_mode* to none.

2.1.3. Determining Technology Libraries for RTL Synthesis

Use the technology libraries associated with the RTL synthesis tool used in your *in-house* synthesis flow for Catapult library characterization. The format of the technology files vary based on the RTL synthesis tool as described in the following table:

Synthesis Tool	File Format
----------------	-------------

Design Compiler	.db
Oasys-RTL	.lib

Table 6: RTL Synthesis Tool Technology File Formats

At this time Catapult can only read technology files in Liberty format (.lib). Availability of the technology library in Liberty format allows for wizard-based automated library creation as well as for built-in prototyping and other features unavailable otherwise.

For best results when creating an ASIC library, you should use the following guidelines for technology files:

- Most Low-Power technology libraries (65nm and smaller) use separate library files for High-Vt and Low-Vt cells along with a common library used by both. Specifying all of the libraries when you create your ASIC library is likely to make the RTL synthesis tool use the Low-Vt cells as they are faster with the same area. We recommend using the High-Vt library for characterization and adding the other libraries later so that they can be used during design synthesis.
- Do not use wire loads unless you understand their effect.
- Once the Catapult library is created, you may like to prune it to remove items that may cause unnecessarily long characterization run times. For example: Remove large multipliers (mgc_mul with an operand width of 64 bit and greater unless your designs are likely to require them). Use this approach for any other operator as long as you are sure the data points will never be needed. Catapult will warn you during design synthesis if it has to use the points beyond the Catapult library characterization range.

2.1.4. Determining Licensing Approach

To ensure a successful library characterization run, you should determine your tool licensing needs and verify their availability as follows:

- **Library Builder** — Full library characterization may take days using a single Library Builder license or overnight using a Library Builder Farm license, so start testing small.

If possible, use the Library Builder Farm option with multiple machines or to connect with a Load Sharing Facility (LSF). Make sure paths and licenses are set up correctly in your shell startup files (eg. .cshrc), so they are found when running on remote machines. For more information, refer to [Using the Library Farm](#).
- **RTL Synthesis Tools** — RTL synthesis tools often require more than a base license to enable the retiming needed to characterize pipelined multipliers, so you should characterize one pipelined multiplier to verify the proper license is available.

Licensed DesignWare for Design Compiler requires a separate license, so you need to verify that it is available for characterization. The DesignWare libraries usually have .sldb extension and should be specified using the synthetic_library variable.

2.2. Creating the Library

The Library Builder provides ASIC and FPGA library templates to help you create and characterize libraries for use in Catapult.

General procedure for characterization:

1. Create a working directory where the new library will be built (a working area).
2. Invoke the Library Builder and set the working directory within the tool. See [Setting the Working Directory](#).
3. Specify the directory path to RTL Synthesis libraries using the **Tools > Set Options > Component Libraries** in the Catapult.

It is necessary to include technology library path(s) so that Catapult Library Builder knows the location of technology files required for characterization. An “unable to locate” error will be observed during characterization when the path(s) is not specified. Note that DesignCompiler supports .db format whereas Catapult can only read technology files in Liberty format (.lib) so it is recommended to include path(s) to both file sets as they are often located in different file system directories.

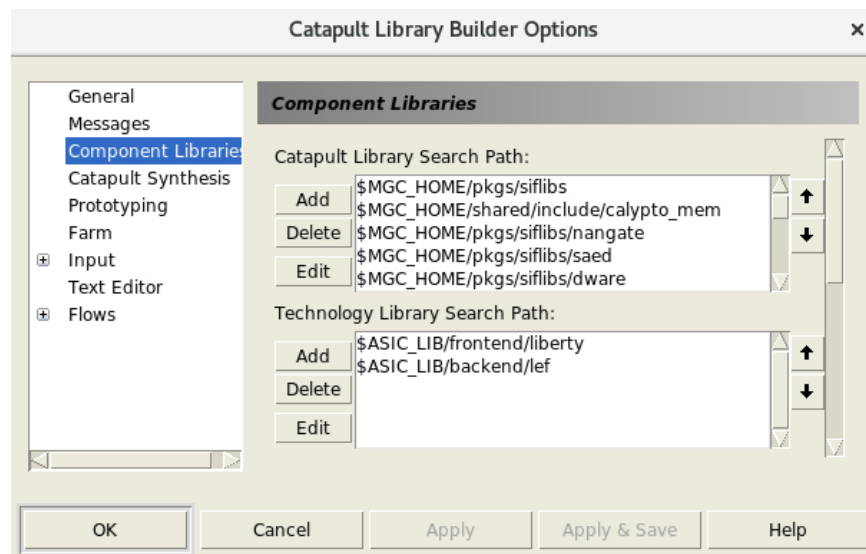


Illustration 5: Option for the Technology File Search Path

To include technology libraries, click under Menu: Tools > Set Options > Component Libraries. Add “Technology Library Search Path” and include path(s). Apply and OK.

4. Verify RTL Synthesis Tool Settings

The settings used for library characterization must match the RTL synthesis settings used in downstream synthesis flow. If the synthesis flow used for characterization differs from your standard RTL synthesis flow, the area and timing estimates in Catapult are likely to be inaccurate.

You can view the RTL Synthesis tools setting in the Flows Option within Library Builder. (eg. Tools > Set Options > [+] Flows > Design Compiler).

It is important to specify your DesignCompiler tool location under “executable path”. “Downstream synthesis failed invocation” error maybe observed during characterization if the path is not specified. The executable path can also be specified by using the command:

```
options set Flows/DesignCompiler/Path type_your_tool_path_here
```

Note: If using small process technology <7nm with Design Compiler, then use **dcnxt_shell** otherwise set the executable name to **dc_shell**.

Note: Control of flows can be set as Flow Options, or as Library VARs(using Library Wizard), or a mixture of the two approaches. Flows Options override Library variables.

5. Create a new library from one of the supplied templates. Library Builder provides the following methods:
 - [Creating a New Library with the Library Characterization Wizard](#) (requires technology library in Liberty format).

The easiest way to get started making a new Base ASIC Catapult library for your technology is to use the Library Characterization Wizard. This wizard allows you to select one (or more) Liberty files representing the operating conditions and characteristics of your desired technology library. After selecting the target operating conditions and the downstream RTL Synthesis tool, the wizard automatically reads the Liberty file and extracts key pieces of data necessary to setup the Catapult Base Library.

- [Command line library creation](#)
6. [Characterize the library.](#)
 7. [Save the library.](#)

Setting the Working Directory

The Library Builder works with the files, in a working directory, which you create before you invoke the tool. This is also the place where all generated output files are placed.

1. Click on the **Set Working Directory** task in the Task Bar window or use the **File > Set Working Directory...** menu item to open file system browser.
2. Browse to the directory that you want to set as the working directory and click **OK** to make it your working directory.

Alternatively, you can use the [set_working_dir](#) command.

Command line library creation

NOTE: It is highly recommended that you use the [Library Wizard](#) to setup the library characterization process in Library Builder.

Library wizard and the dialog boxes are two ways of specifying the data to set up the library. Both of them eventually compose and call the library creation command with the library setup passed using command switches.

The library creation command [flow run](#) launches the *library add flow* for the specified RTL synthesis tool. Library Builder provides flow packages for each of the supported synthesis tools.(eg *DesignCompiler*). Run the flow for the target synthesis tool and specify a library template and its required settings. The flow will add the library to the Library Builder database.

The command format is:

```
flow run /<package>/library add <lib_template> <options>
```

For example, the following command creates a Base ASIC Library (base) for the Design Compiler tool.

```
flow run /DesignCompiler/library add base \  
-libname my_lib \  
-libtitle my_lib_title \  
-vendor Sample \  
-technology 180nm \  
-libs_liberty sample_180nm.lib \  
-libs_db sample_180nm.db
```

The set of valid options varies depending on the type of library template. To get a list of the valid template names for a flow package, use the “-help” switch as follows:

```
flow run /<package>/library add -help
```

Similarly, to see the set of valid options for a particular library template, use the following command:

```
flow run /<package>/library add <lib_template> -help
```

Some customers preferred the script based approach as opposed to the manual GUI based library creation. For base library creation we discourage this practice because new library settings get added from one release to another thereby invalidating the scripts or making them incomplete. The biggest problem with this approach is that no checking against the technology library is done making the process error prone.

Creating a New Library manually

NOTE: It is highly recommended that you use the [Library Wizard](#) to setup the library characterization process in Library Builder.

Catapult Library Builder provides templates for the various types of libraries you can create. The Base ASIC Library template contains all of the basic components needed for synthesis. There are various templates for creating ROM, Custom RAM and RegisterFile libraries. Finally, the Blank Library template is an empty library that you populate with custom component modules and operators.

Base ASIC libraries should be created using the Library wizard. When manually creating a base library you would have to enter all the data set up by the wizard manually. This method is not recommended since no

checking/suggesting is done by Catapult thus making this process more error prone. This method should only be used if the technology file in Liberty format is not available. Note that some of the library variables created by the library creation wizard cannot be entered through dialog boxes. Such variables must be created after the library creation as described in [Editing Parameters](#).

The procedure for creating a library by using the GUI is as follows:

1. Click the **New Library** task in the Task Bar window, or select the **File > New > New Library** menu item to open the Library Creation dialog box.
2. In the Library Creation dialog box, select the synthesis tool to be used for characterization. For ASIC libraries, choose either **Synopsys Design Compiler** or **Siemens Oasis-RTL**.
3. Select the library template under the selected synthesis tool.

Fill in the fields on the right side of the Library Creation dialog box. Items in red are required. If you have any questions about any of the entries, refer to the online Help for each form by clicking the **Help** button.

4. Click OK. The new library is created in the Library Builder database but not saved to disk until you explicitly save it.
5. Edit the library if necessary. In the Library Explorer window, double-click on the newly created library to open the Library Editor. (or right-click and select **Edit** from on the pop-up menu).

2.3. Creating a New Library with the Library Characterization Wizard

After you click the **Library Wizard** task in the *Task Bar* window, the Wizard will present the following panes to guide you through the setup of your Catapult library.

1. [Technology Specifications](#)
2. [Settings \(1\): Catapult GUI and Library Modules](#)
3. [Settings \(2\): RTL Synthesis Libraries and Library Module Usage](#)
4. [Settings \(3\): Optional Files/Settings for other RTL Synthesis flows](#)

After the Wizard generates the library template for your technology, you can characterize the library components using your RTL Synthesis tool.

Technology Specifications

In the Library Setup Wizard, the first pane allows you specify the liberty files for your target technology. Library Builder parses the liberty file to extract default values for the Catapult library characterization process.

1. Click on the **Add** button and navigate to your Liberty file(s). Make sure you add all Liberty files in your technology library. In the screen shot below, the file *NangateOpenCellLibrary_typical.lib* was added:

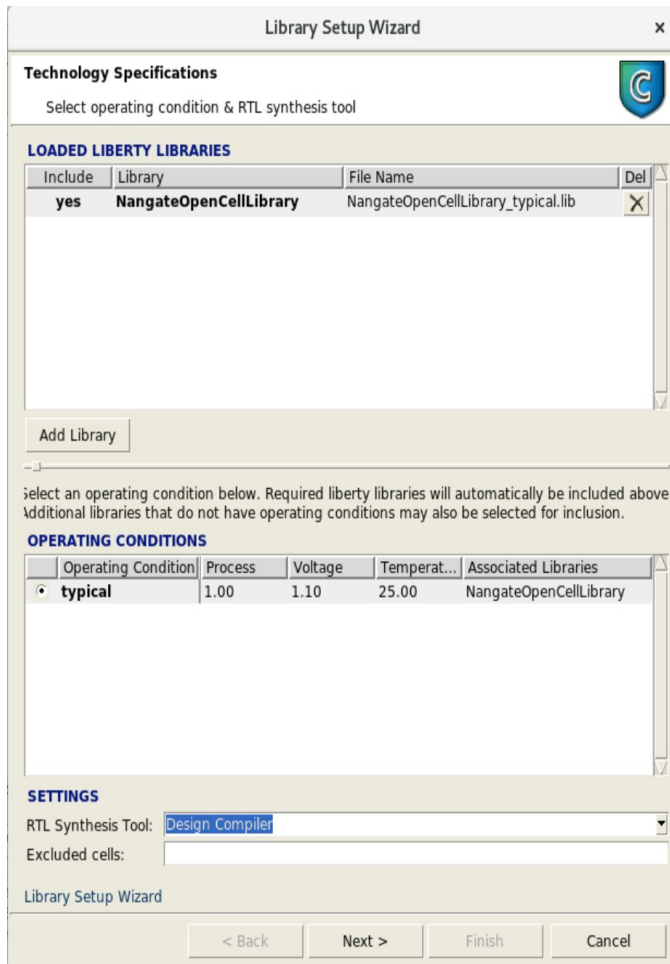


Illustration 6: Library Wizard: Loaded Liberty Library

Sometimes there will be variations in the extension for Liberty files, such as *.lib_ccs_tnv. Often these come from the IP vendor in compressed form (Such as .lib.gz). Library Wizard currently does not support compressed files and hence only .lib is found (instead of variations) and the wizard will not load compressed Liberty files. Since the wizard shows only .lib, you can include the available .lib (with no variation) for now and modify it later in the Library Explorer.

2. Select the Operating Conditions for the new Catapult Library by checking the radio button next to the operating condition name. For reference, the Process, Voltage and Temperature for that operating condition are extracted from the Liberty file and displayed in the table. Only one Operating Condition can be selected for the library.

Note that the set of loaded libraries containing the selected operating conditions are selected (marked yes in the Include column). At this point you can also include the libraries that do not have any operating conditions specified. NOTE: The wizard does not allow you to select libraries that have operating conditions specified, but lack the selected operating conditions.

3. Select the downstream RTL Synthesis tool. This tool will be used for characterization. If the completed library needs to support other RTL synthesis tools in the downstream flow, you can add those tool names to the *Flows* section of the library after characterization.
4. Specify the cells to be excluded – The Library Builder generated script for your RTL synthesis tool will set the *dont_use* attribute of this list of cells. Cells specified in this list will also not be considered as driving cells or external load. On this wizard entry, the excluded cells have to be specified in the <library name>/<cell name> format. The wizard will check the specified cells against the technology library and only the cells found in the selected libraries will appear in the **Excluded cells** box on the next page.
5. Click **Next**

Settings (1): Catapult GUI and Library Modules

Now the wizard selected the appropriate defaults for setting up a Catapult Library for characterization from the libraries you selected and taking into account the selection of the RTL synthesis tool selection and excluded cells. These defaults are then shown on this and the following wizard pages to enable you to review them and make changes if necessary.

Library Setup Wizard

Settings (1)

Catapult Library Settings

Library name: NangateOpenCellLibrary_dc
Library title: Catapult base lib for NangateOpenCellLibrary typical
Vendor: VENDOR
Technology: technology

☒ Default selected
☐ Minimize characterization points
☒ Include Advanced Components
☒ Advanced Retiming
☐ No Boundary Optimization
☐ Enable Scan Registers

Technology Library Files

Liberty Library Files: NangateOpenCellLibrary_typical.lib
Synopsys DB Library Files: NangateOpenCellLibrary_typical.db
PowerPro Library Files:

Design Compiler Settings

☐ Set Compile Mode to Ultra
DesignWare Support: none
Synthetic Library(ies): dw_foundation.sldb standard.sldb

Characterization Settings

Clock Period Fastest: 0.001
Clock Period Slowest: 1000.0
Clock Period Percents: 100% 75% 50% 0%
Timing Report Fractional digits: 7

Library Setup Wizard

< Back Next > Finish Cancel

Illustration 7: Library Wizard: Selecting Operating Conditions and RTL Tool

The first **Settings (1)** page allows you to name the library and control how it is displayed in Catapult.

In the **Catapult Library Settings** section, the *Library name*, *Library title*, *Vendor* and *Technology* fields on this pane are arbitrary names that are only used within the Catapult GUI in the Libraries Constraint Editor. By default, Library Builder derives the name based on the Liberty file and the selected synthesis tool. In this

example, the library name is *gscl45nm_or* (eg. characterizing the gscl45nm using Oasys RTL). Although this naming will not affect library characterization, it is advised to enter a meaningful name for better identification.

If the **Minimize characterization** points box is selected, Library Builder will create the library with the minimal set of characterization points on some operators. This approach usually reduces the characterization time with some possible accuracy trade-off. This option should be selected only if characterization run time is an issue and accuracy/correlation is not important. Refer to [Reducing Characterization Time](#), for more information.

In the **Technology Library Files** section, you can specify the names of the Technology Library Files. **DO NOT include the directory paths; just filenames.** Library Builder uses the Technology Library Search Path in the Component Libraries option to specify the directory paths to these library files. For characterization, you only need to specify the library files that are required for your RTL synthesis tool. This section includes the technology files used by all supported RTL synthesis. This page also displays the Liberty files you selected on the previous page and allows you to specify the rest of RTL synthesis library files associated with this technology (.db, .lef etc). For example, if you are using Design Compiler, you will need to specify the .db files.

The <RTL Synthesis Tool> Settings section allow you to specify library specific options that are supported in the corresponding automated RTL Synthesis flow in Catapult. The number of available setting will vary depending on the RTL synthesis flow.

NOTE: *It is important that these values match the values used in your RTL Synthesis scripts.* For more information on these fields, refer to the [Basic RTL Technology Settings](#) section.

Settings (2): RTL Synthesis Libraries and Library Module Usage

The second **Settings (2)** page allows you to review the technology settings that Library Builder uses to to setup the designs for characterization including the operating conditions, physical units, timing and driving cell information, etc.

Illustration 8: Library Wizard: Library Settings

In the **Technology Preferences** section, you can provide additional information to assist with characterization and downstream flows. The Library Wizard derived the default values from the liberty files. If your RTL synthesis flow excludes any cells from the technology library, then you should specify the cell name in a space-separated list in the **Cells to exclude** field. Catapult Ultra allows you to optimize your RTL netlist using the PowerPro engine. In this flow, Catapult builds a clock tree for timing estimation, you can specify the **default Clock Buffer** for this flow.

In the **Library Settings** section, Library Wizard extracts this information from the liberty files. You rarely need to adjust these values.

The **External Driver Cell** field will pre-populate the a list of registers from the liberty file. You should select a cell that represents the typical loading of an operator in your designs.

Settings (3): Optional Files/Settings for other RTL Synthesis flows

The third **Settings (3)** page has allows you to specify the optional technology files and tool settings for RTL synthesis tool flows that may be used with the Catapult library. Although these synthesis tools will not be used for characterization, Catapult users may use this library with other RTL synthesis tools in their design flow. If you set these values during library creation, users will not have to modify the Catapult library with these value later if they need to use another RTL synthesis tool.

Library Setup Wizard

Settings (3)

Optional Technology Files

LEF Files: nangate45nm.lef

Oasys Process Technology Files:

Oasys Layer Map Files:

Aprisa Technology File:

Aprisa RLC Min File:

Aprisa RLC Max File:

Capacitance Table Files:

Optional Aprisa-RTL Settings

Aprisa Parasitic Cond Min:

Aprisa Parasitic Cond Max:

Aprisa Chip Utilization: 40%

Optional RTL Compiler Settings

☐ Set Compile Mode to Placed

☐ Disable Retiming

Wire Loads (for DC/RC/Oasys)

Interconnect Mode: none

Wire Load Selection Group:

Wire Load Mode: enclosed

Wire Load Model:

Library Setup Wizard

< Back Next > Finish Cancel

Illustration 9: Library Wizard: Characterization Settings

Specifying the other files allows Catapult users to utilize the automated flows to other RTL synthesis tools. You can always add these files to the library after characterization. For each field on this page, the Wizard denotes the RTL synthesis tool flow that uses the file. Similar to previous library file entries, you should only specify the filename of the technology file, not the directory name. The Technology Search Path option defines the directory search path for these technology files.

If you are satisfied with the settings, click **Finish** to complete the process and create the Catapult Library. You can now proceed to the library characterization process described in [Characterizing Libraries or Components](#).

2.4. Characterizing Libraries or Components

Characterization of a Catapult library is the process of synthesizing a pre-defined set of components, using the RTL synthesis tool of choice, in order to obtain their area and timing characteristics. Library Builder manages the characterization runs with the RTL synthesis tool and stores the area/delay values in the library. Catapult uses these values for optimization and scheduling.

The Catapult *Base ASIC Library* template consists of about 25 modules (MODs). are characterized at multiple performance points, producing a robust library with which Catapult can deliver highly accurate area estimates.

NOTE: The library characterization process uses one of the supported RTL synthesis tools to synthesize a sufficient number of the qualified modules (QMODs) in a library in order to obtain a representative sample of the area and timing characteristics of the QMODs. Based on the actual measured values in the sample data set, the Library Builder uses curve fitting algorithms to estimate the characterization values for all other

QMODs. Each QMOD is measured at various clock speeds based on the multi-point characterization settings for the library. Refer to [Determining Characterization Points](#) for more information.

A *module* (MOD) is a library component implementing one of the operators available to the scheduler. For example the MODs `mgc_mul` and `mgc_mul_pipe` (multiplier and pipelined multiplier) implement the functionality of the multiplication operator MUL.

A QMOD is module with all characterization parameters explicitly specified. For example `mgc_and (1,2)` is a qualified configuration of `mgc_and` with one output and two inputs (a single AND2 gate).

Library characterization consists of the following steps:

1. In the Library Explorer window, select the items to characterize. If the top level is selected, the entire library is characterized. Expand the tree to select individual MODs, QMODs or QMOD datasets.
2. Spot Checking Library Settings
3. Launch the characterization process on the selected items. See Running Characterization.
4. After characterization is complete, use the Plot window, reports and transcript files to view and analyze the characterization data and resolve any failures.
5. If necessary, modify the interpolation equations of individual MODs to refine the correlation between the estimated characterization values and the measured values obtained from the RTL synthesis tool. Then recharacterize MODs. Refer to [Evaluating Library Characterization Results](#) for more information.
6. If errors occurred during characterization, use the characterization transcript to [diagnose the problem](#).

2.4.1. Spot Checking Library Settings

After the library is created we suggest spot checking the library before running full characterization:

- Characterize a single inverter first. Make sure that, for the Fastest component, the reported Slack is negative. If it is not reduce the value of the `characterize_clock_period_fastest` variable.
- Characterize the largest multiplier and verify that the Smallest component has positive Slack reported. Otherwise increase the value of the `characterize_clock_period_smallest` variable.
- Characterize a pipelined multiplier to verify that retiming works properly.

2.4.2. Running Characterization

During the characterization process, the Catapult Library Builder commands are displayed in the Transcript window, giving you a record of the entire session. You can scroll through the transcript and print or save it.

Once the library is loaded into the Library Explorer window, right-click on the object you want to characterize and choose the **Characterize** command. The characterize command operates on all objects hierarchically contained under the selected object. For example, if a module is selected, each qualified module it contains will be characterized. Similarly, if the library object is selected, the entire library will be characterized.

Select **Characterize** on the Task Bar to characterize all libraries in the Library Explorer window. Illustration 10 shows how to initiate the characterization command.

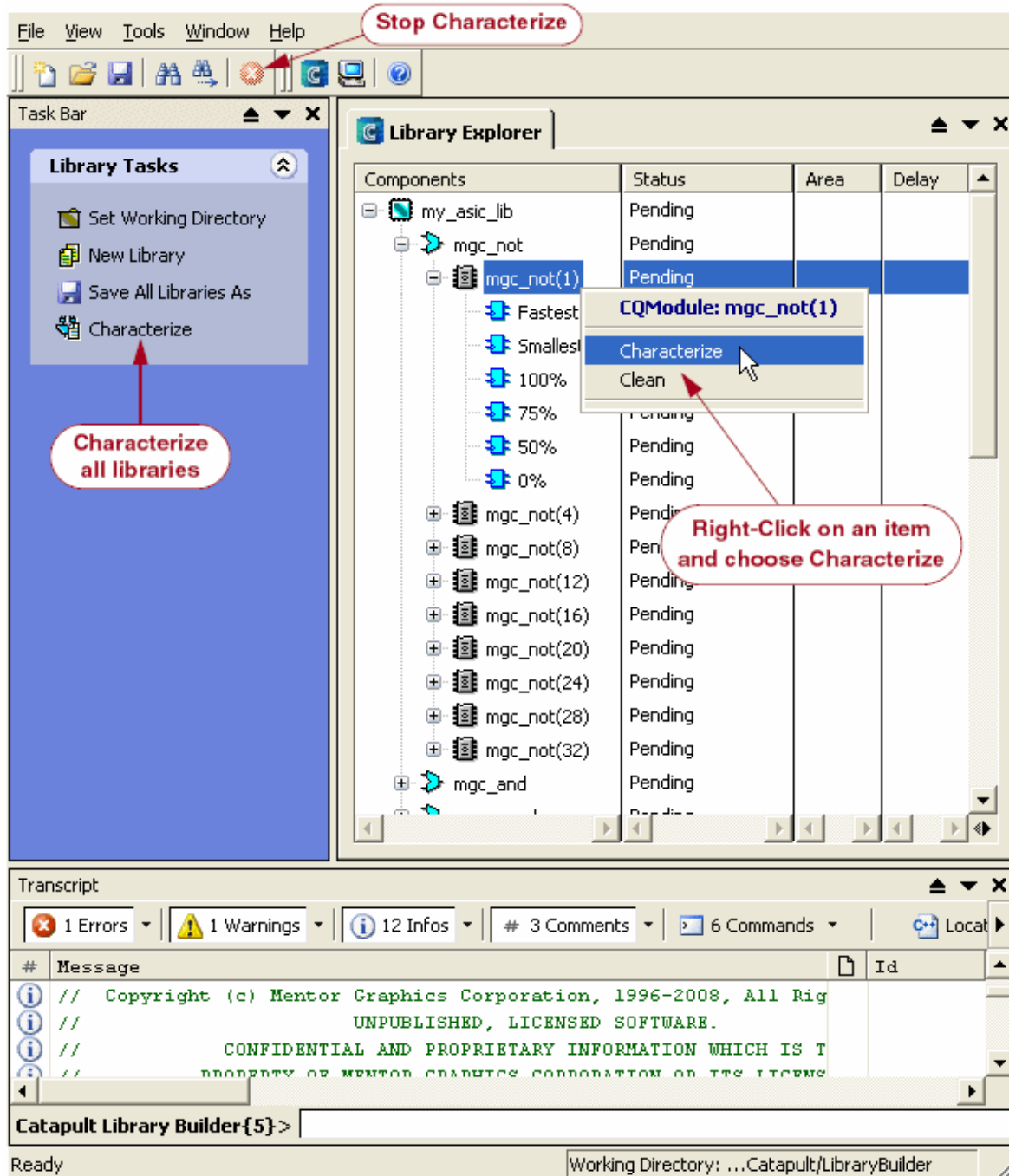


Illustration 10: Characterizing the Library or Component

As the characterization process runs, it displays information about its progress in the Status column of the Library Explorer window, as well as in the Transcript window. A full transcript of the characterization task is automatically saved in the project directory.

To stop a characterization job in progress, click on the **Stop** button on the tool bar. It usually take a minute or so for the job to terminate. After it terminates, the component(s) that were being characterized will have a **Failed Generate** status.

When characterizing a large number of components, the Library Builder queues a series of characterization tasks, and each task will contain a multiple components. The maximum number of components that can be grouped into a task is configurable. To adjust the default setting, edit the **Maximum Components Per Task** option (Refer to Set General Options. **Tools > Set Options > General**).

Illustration 11 shows an example of the how the Library Explorer window might appear while a characterization job is in progress. For this example, all of the *mgc_equal* components are being characterized, and the *Maximum Components Per Task* option has been set to four. The figure shows the first group of four components is in progress, indicated by the *Generating* status. The remaining *mgc_equal* components have a *Queued* status, and will be processed in turn.

Components	Status	Area	Delay	Clockperiod	Minclkprd
mgc_reg_neg(32,1)	Passed				
Fastest	Passed	1139.1	0.38	0.001	0.0
Smallest	Passed	1139.1	0.38	100000.0	0.0
100%	Passed	1139.1	0.38	0.38	0.0
75%	Passed	1139.1	0.38	0.38	0.0
50%	Passed	1139.1	0.38	0.38	0.0
0%	Passed	1139.1	0.38	0.38	0.0
mgc_equal	Generating				
mgc_equal(4)	Generating				
Fastest	Generating			0.001	
Smallest	Generating			100000.0	
100%	Queued				
75%	Queued				
50%	Queued				
0%	Queued				
mgc_equal(8)	Generating				
mgc_equal(12)	Queued				
mgc_equal(16)	Queued				
mgc_equal(20)	Queued				
mgc_equal(24)	Queued				
mgc_equal(28)	Queued				
mgc_equal(32)	Queued				
mgc_decode	Pending				
mgc_shift_l	Failed Generate				
mgc_shift_r	Failed Invocation				
mgc_shift_bl	Pending				

Illustration 11: Characterization Status Information

Illustration 11 also shows some of the other status values for characterization, such as *Pending*, *Queued* and *Failed Generate*. The following table defines the meaning of the status values that may appear in the **Status** column:

Status	Description
Pending	Component currently has no characterization data and is not currently queued for characterization.
Queued	Component is in the queue for automatic characterization.
Configuring	Initial state of the automatic characterization process which configures the design

	constraints and generates the characterization task script.
Generating	Characterization task is running. The component is being synthesized and the resulting area and timing characteristics are being collected.
Passed	Characterization task completed successfully and area/timing data was obtained.
Failed Analyze	An error was encountered when parsing the transcript to collect timing and area estimates for the design.
Failed Generate	Characterization task returned a non-zero exit status or did not obtain timing and area information.
Failed Invocation	The downstream synthesis tool failed to invoke.

Table 7: Component Characterization Status

2.4.3. Determining Characterization Points

During the characterization process, Catapult Library Builder collects multiple sets characterization data (area and timing) for each component. You can think of each data set as a point along an area versus delay graph of the component. For example, Illustration 12 shows how the graph of a 16-bit Adder might look.

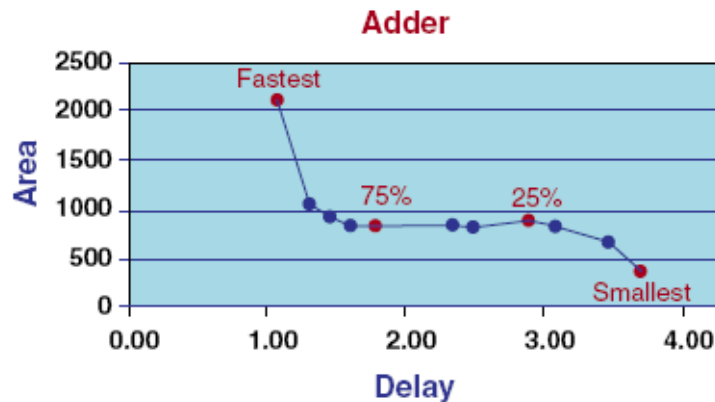


Illustration 12: Adder Characteristics Graph

Typically the characteristics of the two end points (Fastest and Smallest) are rarely used in designs. So the Library Builder allows you to specify multiple points between the end points at which area and timing data will be collected. The target points are specified as percentages of the range bounded by the fastest and smallest points.

By default, four data sets are collected and their distribution is: 100%, 75%, 50% and 0% (*Fastest* = 100% and *Smallest* = 0%). You can specify a different number of data sets and/or different percentages when creating a new library. Library Builder limits the number of data sets that you can specify to 15 values. You can edit the data set values in the *Clock Period Percents* field of the *Advanced* tab in the Library Creation dialog box as shown in Illustration 13. Refer to [Creating a New Library manually](#) for detailed information.

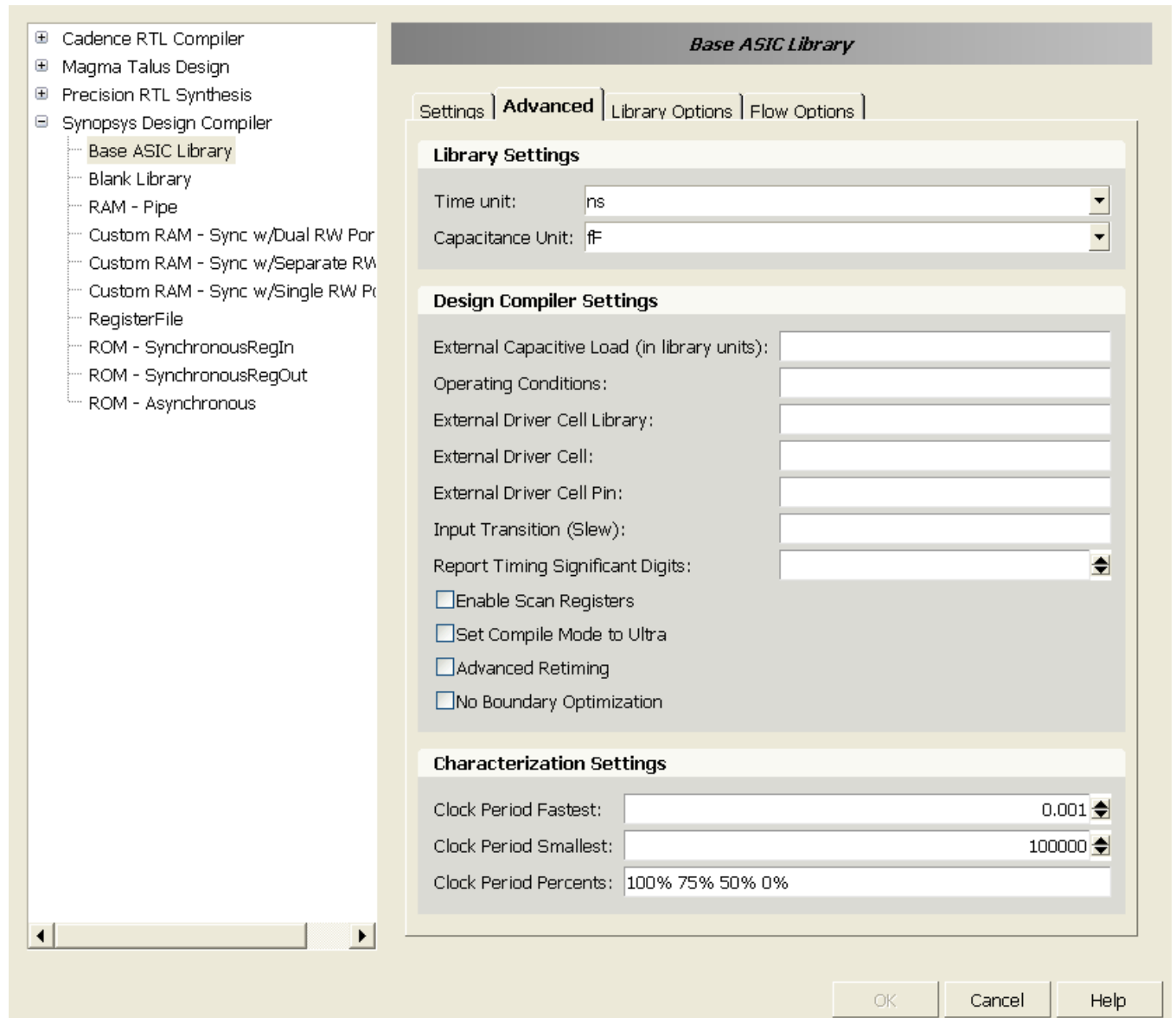


Illustration 13: Multi-Point Characterization Settings on Library Creation Dialog

The characterization process works as follows for each component:

- Obtain fastest data set:
Run the synthesis tool with a target delay constraint specified by the Clock Period Fastest setting. (default is 0.01 ns.)

NOTE: The clock periods must be able to generate the fastest design, i.e. the period must be small enough to constrain a simple logic gate such that it does not meet timing. If the tool is able to easily meet timing the curve described on the previous page will not be accurate.

- Obtain *smallest* data set:
Run the synthesis tool with a target delay constraint specified by the *Clock Period Smallest* setting. (default is 100,000 ns.)

- Obtain each intermediate data set:
Use the actual delay values obtained from the fastest and smallest characterizations to calculate the target delay constraints to be used for the intermediate characterization runs. Then run the synthesis tool for each intermediate point. Intermediate points are specified by the *Clock Period Percentages* setting.

For ASIC libraries, the Library Explorer window displays *Status*, *Area*, *Delay*, *Clockperiod* and *Minclkprd* columns. The *ClockPeriod* column is the clock period to which the design was constrained for characterization. The *MinClkPrd* column is for sequential components, this includes registers. The minimum clock period is the fastest time that the component can be scheduled. The delay is the maximum, input to register, or register to register delay. The Delay column for these components refers to the final register to output delay. The *Minclkprd* column will not appear until a pipelined component is selected in the *Components* column.

When you begin using your library in Catapult, if an operator in a design contains a parameter with a value that lies outside the range within which the corresponding component was characterized, Catapult will list the operator in the *adjust_lib_char.tcl* file in the Catapult solution directory.

2.4.4. Automatic smoothing of ASIC Library Data

During library characterization, Library Builder performs multiple RTL synthesis runs to generate Area/Delay data for parameterized library modules (QMODs) over a range of clock frequencies (datasets). Using this data, Library Builder generates Area and Delay interpolation equations for each library module (MOD) in the library. Catapult uses these equations to make tradeoffs during scheduling. Ideally, the points in the dataset should monotonically change within the dataset. In other words, as the clock period decreases, you would expect the RTL synthesis tool to generate faster implementations. If the RTL synthesis tool returns a non-monotonic point in the dataset, this inconsistency can skew the interpolation equation and cause non-optimal results from the scheduler.

Whenever a library, QMOD, or datapoint has been characterized, re-characterized or if the *adjust_char_library.tcl* is executed, Library Builder and Catapult will identify and modify any QMOD dataset that contains non-monotonic changes. The library always retains the original characterization data so the RTL synthesis information is never lost. This change allows the scheduler to make better decisions and increases the correlation between Catapult and your RTL synthesis tool. If Catapult loads an existing Catapult library that has not been monotonized, a warning will be reported.

This feature is controlled by the following option within Library Builder (by default, it is set to *true*):

```
options set ComponentLibs/SmoothLibData true
```

NOTE: This feature only applies to Catapult libraries for ASIC technologies.

You can revert monotonized library data back to the original characterization estimates using the following command:

```
LB> library check -revert 1
```

To monotonize the library data for an existing Catapult ASIC library, follow these steps:

1. Invoke Library Builder

```
catapult -product lb
```

2. Open the Catapult ASIC library (**File > Open Library...**)
3. Check and address any QMOD monotonic issues in the QMOD's dataset by updating the library

```
LB> library check -update 1
```

4. Save the library (**File > Save Library**)

2.4.5. Queuing Multiple Libraries for Multi-day Run

You can also set up a queue of libraries to characterize as follows:

1. Create all of the new libraries using the steps previously described in this section.
2. Click the **Characterize** task on the *Task Bar* to run the characterization process on all open libraries.

2.4.6. Determining the Fastest Clock Period using Iterative Estimation

The current implementation is a bit of a misnomer, we iterate only once. We do not apply this algorithm to all QMods. The algorithm is applied only on the QMods listed in the `iter_est_qmods` Library Variable. The `iter_est_qmods` is a User-settable Variable. The Library Wizard seeds the values based on datapath bindings from the ASIC Template. The Iterative Estimator is applied only on the Fastest dataset. This feature is only available for DC Library Characterization and OnTheFly DC prototyping with Catapult Ultra.

On the first pass of the IE, the Area/Delay numbers and the clock period is determined for the Fastest Dataset. Timing is not met, which results in a negative Slack number. The clock period is calculated just as it was before, except this time we run another synthesis using that revised clock period, plus an offset (to force DC to try harder). From that iteration a revised Area value and Revised delay value is returned from the synthesis engine. The Area number can be 40% smaller than the Area number from the over-constrained synthesis run and is much closer to reality, as it was validated with a more reasonable clock constraint. However, often this will also result in negative Slack. Remember, these are synthesis runs and they return an estimate. Sometimes a single iteration is sufficient to achieve positive Slack, but often it is not. I've seen some QMods go through 9 iterations and still not meet timing due to misbehavior (or behavior I do not understand) in the synthesis engine.

Some care has to be taken even in this simplistic single iteration algorithm. The User Custom Scripting for analyze and synthesis is right in the middle of this code. As is the writing of the SDC, and loading the technology files. DC is not guaranteed to behave deterministically in this mode, so care must be taken to avoid triggering any bad behavior. Also, loading the technology files, and possibly the associated physical databases for DC-Graphical must be carefully managed because these are very expensive operations. It can literally take hours to characterize a single Module in with DC-Ultra, so we must be mindful of performance with this algorithm.

==How the new implementation of the Iterative Estimator will work ==

The next version of the IE will be just that: iterate more than once. Simply put, it will be implemented in a loop with the number of iterations controlled by a Library Variable, `ieff_iter_max`, default until `ieff_wns_target`

(usually 0WNS) is achieved.

The following list summarizes the library variables that affect iterative estimating. Default values are in parenthesis:

- **ieff_version** (2) Used to ensure the same IEFF method is applied for all existing and any new QMods added via `adjust_char_library.tcl`
- **ieff_slack_multiplier** (0.75) Values close to 1 will converge faster to 0 WNS, but at the risk of missing true fastest implementation. Smaller values push DC to try harder to find that true fastest implementation.

Actual iteration equation: $CCP_{next} = (WNS * ieff_slack_multiplier) + CCP_{previous}$

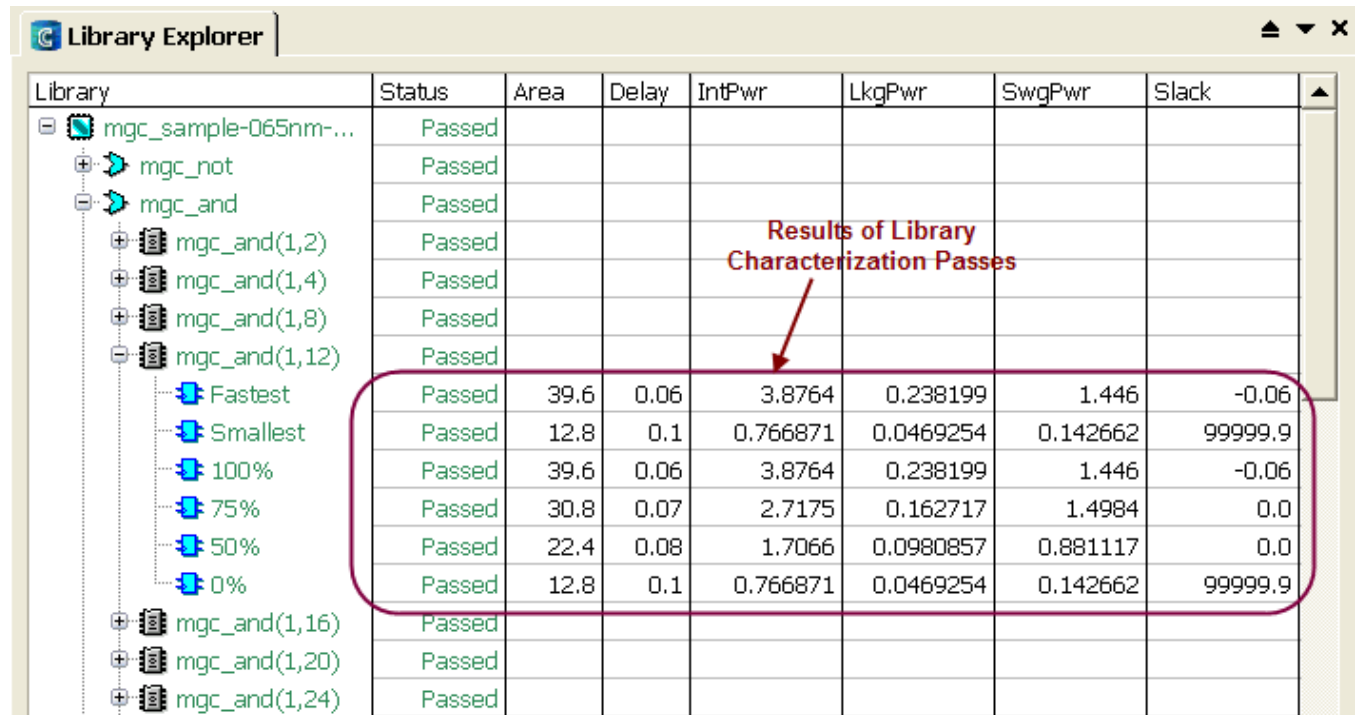
- **ieff_iter_max** (9) Maximum number of times to iterate while converging for 0 WNS
- **ieff_wns_target** (0.000) WNS is a positive number. While 0 is desired, in reality a small number might be just as good should iterate fewer times.
- **iter_est_qmods** (default: all MODs) IEFF will be applied only to modules in this list.

For information on all Catapult library variables, refer the [Editing Library Variables](#) section.

2.5. Evaluating Library Characterization Results

If the library characterization passes, the status is set to *Passed* and the area and timing results display in the Library Explorer windows shown in [Illustration 14](#).

This component was characterized using four-point characterization (default setting). So the display shows separate line items for each target point, plus the fastest and smallest points. Refer to [Determining Characterization Points](#) for information about configuring multi-point settings.



Library	Status	Area	Delay	IntPwr	LkgPwr	SwgPwr	Slack
mgc_sample-065nm-...	Passed						
mgc_not	Passed						
mgc_and	Passed						
mgc_and(1,2)	Passed						
mgc_and(1,4)	Passed						
mgc_and(1,8)	Passed						
mgc_and(1,12)	Passed						
Fastest	Passed	39.6	0.06	3.8764	0.238199	1.446	-0.06
Smallest	Passed	12.8	0.1	0.766871	0.0469254	0.142662	99999.9
100%	Passed	39.6	0.06	3.8764	0.238199	1.446	-0.06
75%	Passed	30.8	0.07	2.7175	0.162717	1.4984	0.0
50%	Passed	22.4	0.08	1.7066	0.0980857	0.881117	0.0
0%	Passed	12.8	0.1	0.766871	0.0469254	0.142662	99999.9
mgc_and(1,16)	Passed						
mgc_and(1,20)	Passed						
mgc_and(1,24)	Passed						

Illustration 14: Characterization Passes

NOTE: If you are characterizing a library using Oasys, it is possible that, when Catapult extrapolates a delay value for operators that far exceed the characterization range (eg. 512 bits on multiplier output), the extrapolated delay value may be zero. You can validate these scenarios by Plotting the Characterization Data. This issue should not occur when characterizing with Design Compiler or AprisaRTL because of improved Interpolation equations and the "Iterative Estimation For Fastest" (IEFF) algorithm.

2.5.1. Delay Characterization Properties

After a successful characterization of a library/component, the Library Explorer window displays delay property values for each data set. These delay properties reflect the maximum values, regardless of the path. Depending on the component type, the following values display:

- **Delay** — If sequential, this value is the register to output value. If combinational, this value is the total input to output delay.
- **ClockPeriod** — Clock period constraint.
- **MinClkPrd** — Minimum clock period is the fastest time a pipelined component can be scheduled. It is either InputDelay + InputSetup, R2RDelay + R2RSetup, or Delay, whichever is largest.
- **Slack** — If sequential, this value is ClockPeriod minus MinClkPrd. If combinational, this value is ClockPeriod minus Delay.
- **InputDelay** — Input to register delay.
- **InputSetup** — Input register setup time.

- **R2RDelay** — Register to register delay.
- **R2RSetup** — Register to register setup time.

Illustration 15 shows how these timing delay values are determined.

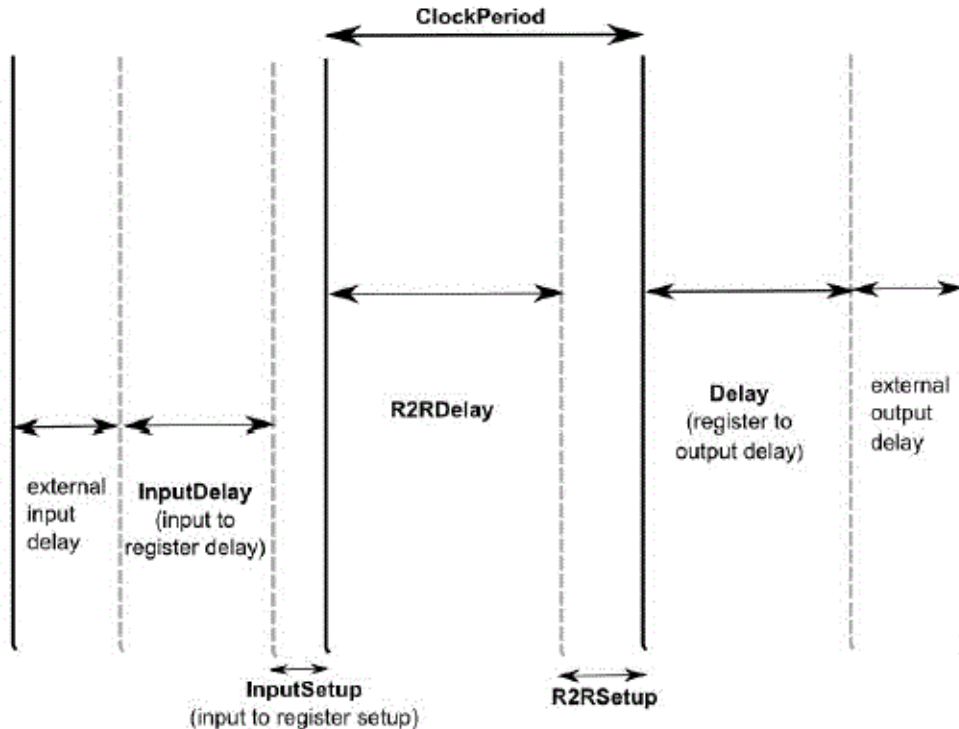


Illustration 15: Sequential Timing Delay Diagram

2.5.2. Resetting the Data Before Another Characterization

Before you can rerun characterization on a library that has passed characterization, you must clear the values. Right-click on the target object and select **Clean** from the popup menu. However you can recharacterize a single component without having to clean it beforehand.

Once the values are removed from the Area and Timing fields, right-click on the target object and select **Characterize** from the popup menu.

2.5.3. Exporting the Characterization Datapath

In some cases, it may be easier to evaluate the library data in another tool. To facilitate this flow, you can export the characterization data from Library Builder to a file in the CSV format.

```
LB> library check -dump2csv 1
```

2.5.4. Plotting the Characterization Data

The Plot window provides a graphical view of the interpolated and measured characterization values for a user-specified range of qualified modules (QMODs) in a library module (MOD). The user interface of the Plot window allows you to dynamically plot different ranges of QMODs by selecting different combinations of properties and parameter values. It also allows you to add, remove and characterize QMODs.

As shown in Illustration 16, the Plot window is accessed from the Library Explorer window by right-clicking on a MOD and selecting Plot from the popup menu. The main area of the Plot window contains a graph of the characterization data. The red and gold circles on the graph represent characterized QMODs, their *measured* values obtained from the downstream RTL synthesis tool. The blue circles are estimated characterization values for valid QMOD configurations that do not yet exist in the MOD. The gold color circle indicates the QMOD whose measured value deviates the most from the Library Builder estimated value for it. In other words, the gold QMOD has the largest margin of error (Max Error) relative to all other measured QMODs on the graph.

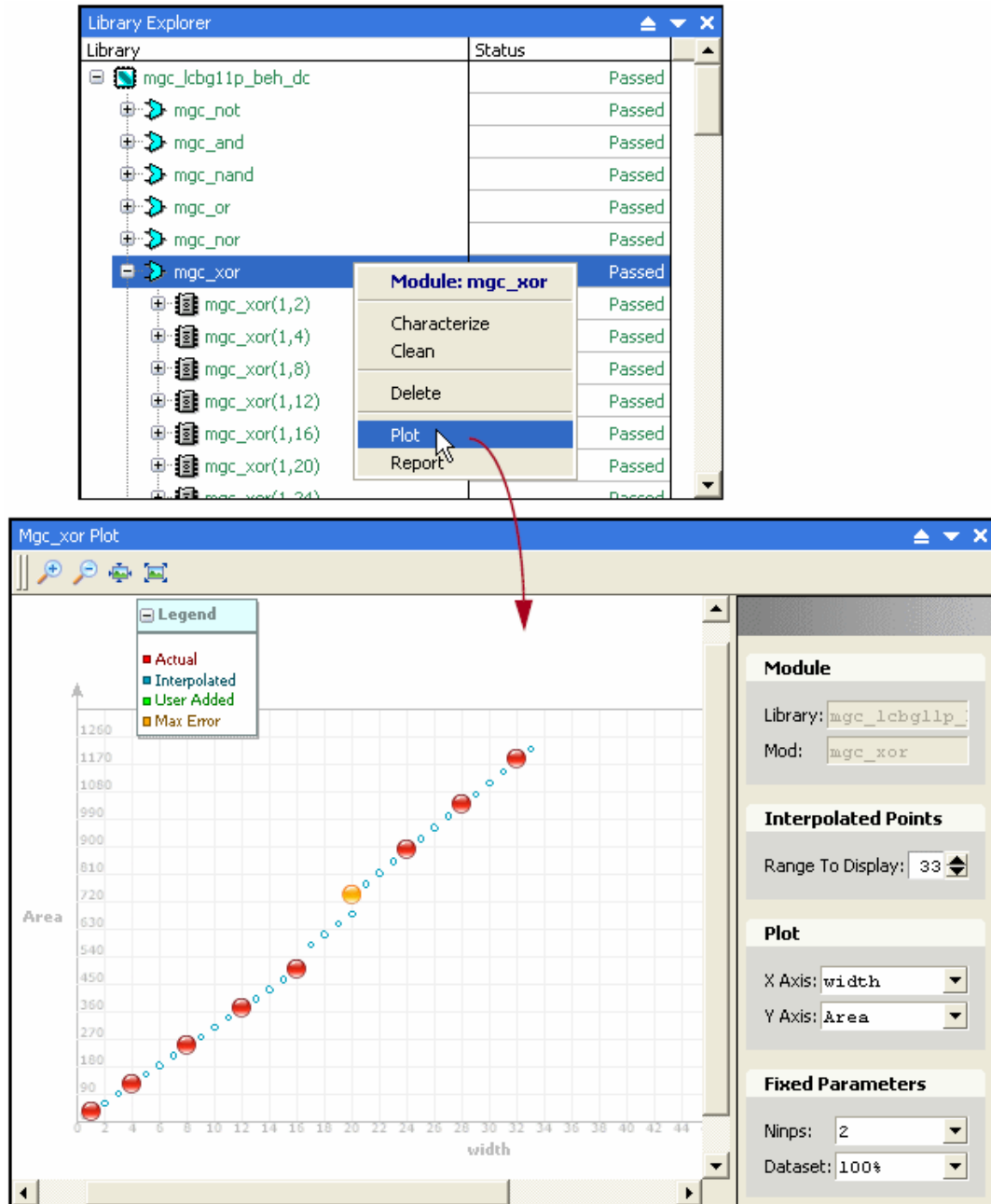


Illustration 16: Opening a Plot Window for the mgc_xor Module

The Y-axis is the range of values for one of the MODs characterization properties. The X-axis is the range of valid values for one of its parameters. All other parameters are set to fixed values within their respective valid ranges. By default, the first property listed in the MOD's All bindings is the initial Y-axis setting, and the first parameter listed is the initial X-axis setting. Similarly for each fixed parameter, the first valid value in its range is the default setting.

Use the interface panel on right side of the window to plot different configurations. For example, using the drop-down menu of X-Axis field, select the *ninps* parameter. The plot automatically updates to graph *ninps* on the X-axis and *width* as a fixed value, shown in Illustration 17.

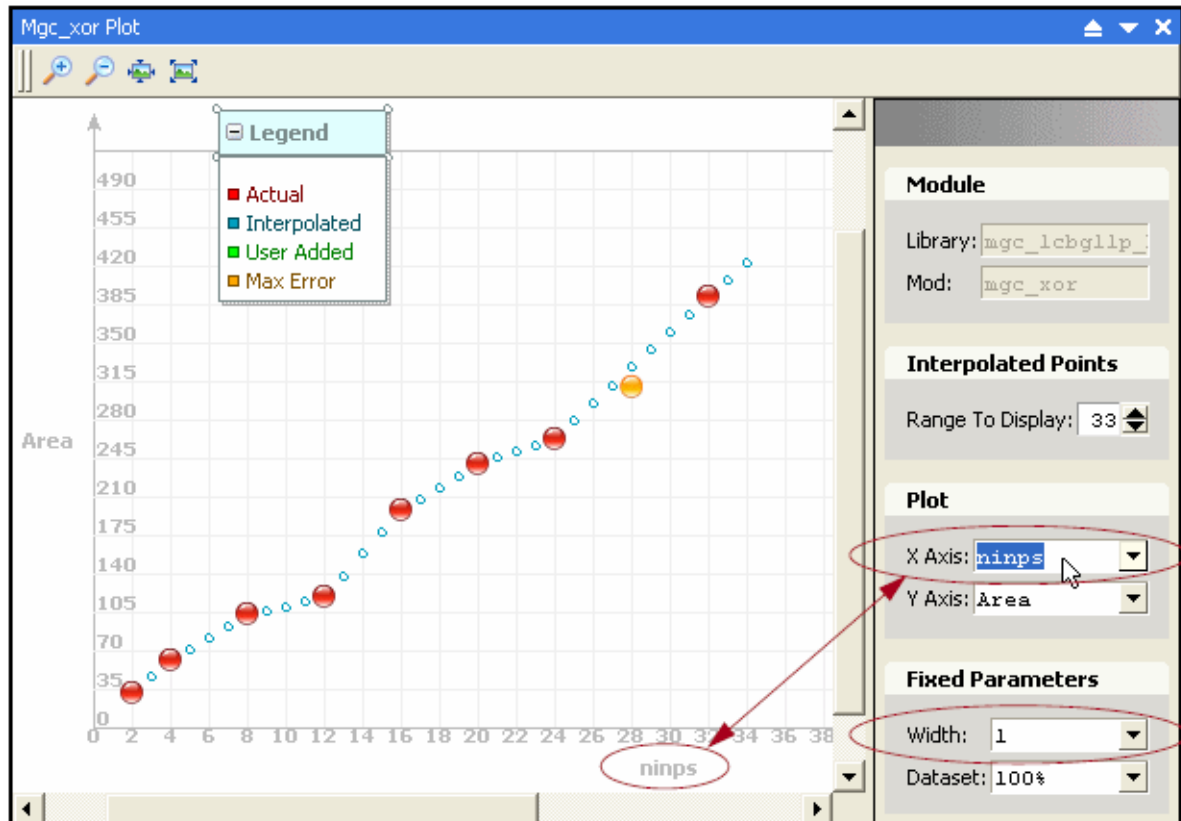


Illustration 17: Updated Plot with Ninps along X-Axis

In Illustration 17, of all the QMODs in *mgc_xor* MOD, only the following nine measured QMOD configurations are plotted because they are the only ones that match the *Plot* and *Fixed Parameters* settings. The first parameter, width, is fixed at the value 1, and all values of the *ninps* parameter are plotted along the X-Axis:

```
mgc_xor(1,2)      mgc_xor(1,20)
mgc_xor(1,4)      mgc_xor(1,24)
mgc_xor(1,8)      mgc_xor(1,28)
mgc_xor(1,12)     mgc_xor(1,32)
mgc_xor(1,16)
```

The *Range To Display* field allows you to change the number of estimated data points (blue circles) that are plotted. Generally, the *Range To Display* number should be close to the X-axis value of the highest measured QMOD. For example, in Illustration 17, the *Range To Display* is 33, and the highest *ninps* value in the range of QMODs is 32 (*mgc_xor*(1,32)).

2.5.5. Adding/Removing QMODs

You can use the Plot window to add new QMODs to the MOD by right-clicking the blue circles and selecting **Add QMOD** (or *Add and Characterize QMOD*) from the popup menu. The new QMOD will be immediately added to the library and have the same parameter values as the selected blue circle. If it is added but not characterized, it is represented on the graph as a green vertical line (unknown Y-Axis value). After it is characterized, it will become either a red or gold circle.

Illustration 18 illustrates the procedure by adding the new QMOD `mgc_xor(12, 28)`. The Library Builder automatically updates the QMOD list in the Library Explorer window and in the module's *Qmods* section in the Library Editor window.

To remove a QMOD, right-click on it and choose **Remove QMOD** from the popup menu.

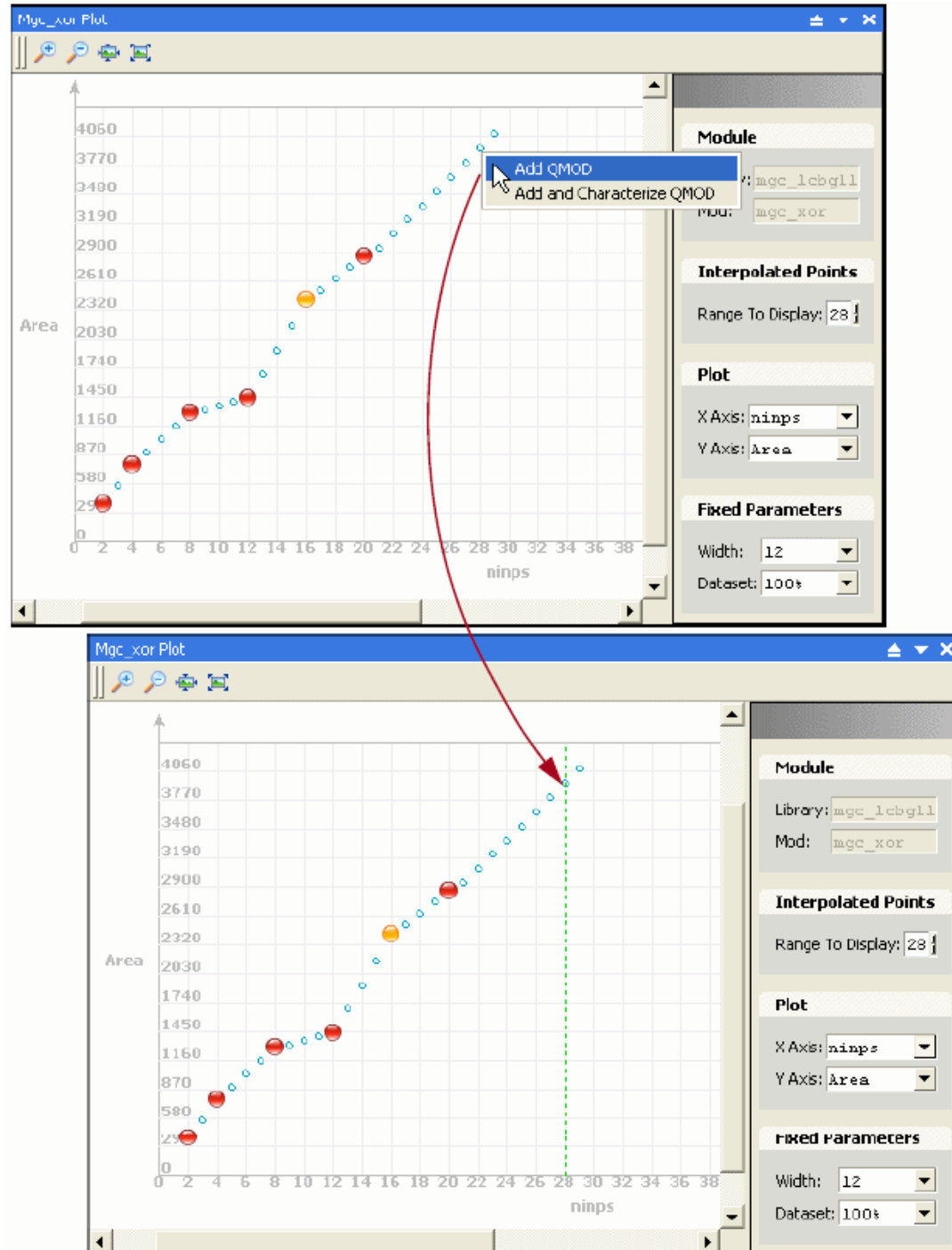


Illustration 18: Adding QMOD from the Plot Window

2.5.6. Viewing the Characterization Transcript

Once the library characterization starts, progress and commands being run are displayed in the Transcript window. Double-click on a component (or right-click and select View Transcript as shown in Illustration 19) to see a transcript of the commands and tasks performed by Catapult Library Builder.

If you run a *grouped* job, the transcript will collapse certain sections so that only the most relevant part are visible. It may be desirable to see others by expanding the +/- hierarchy points, or use the expand all.

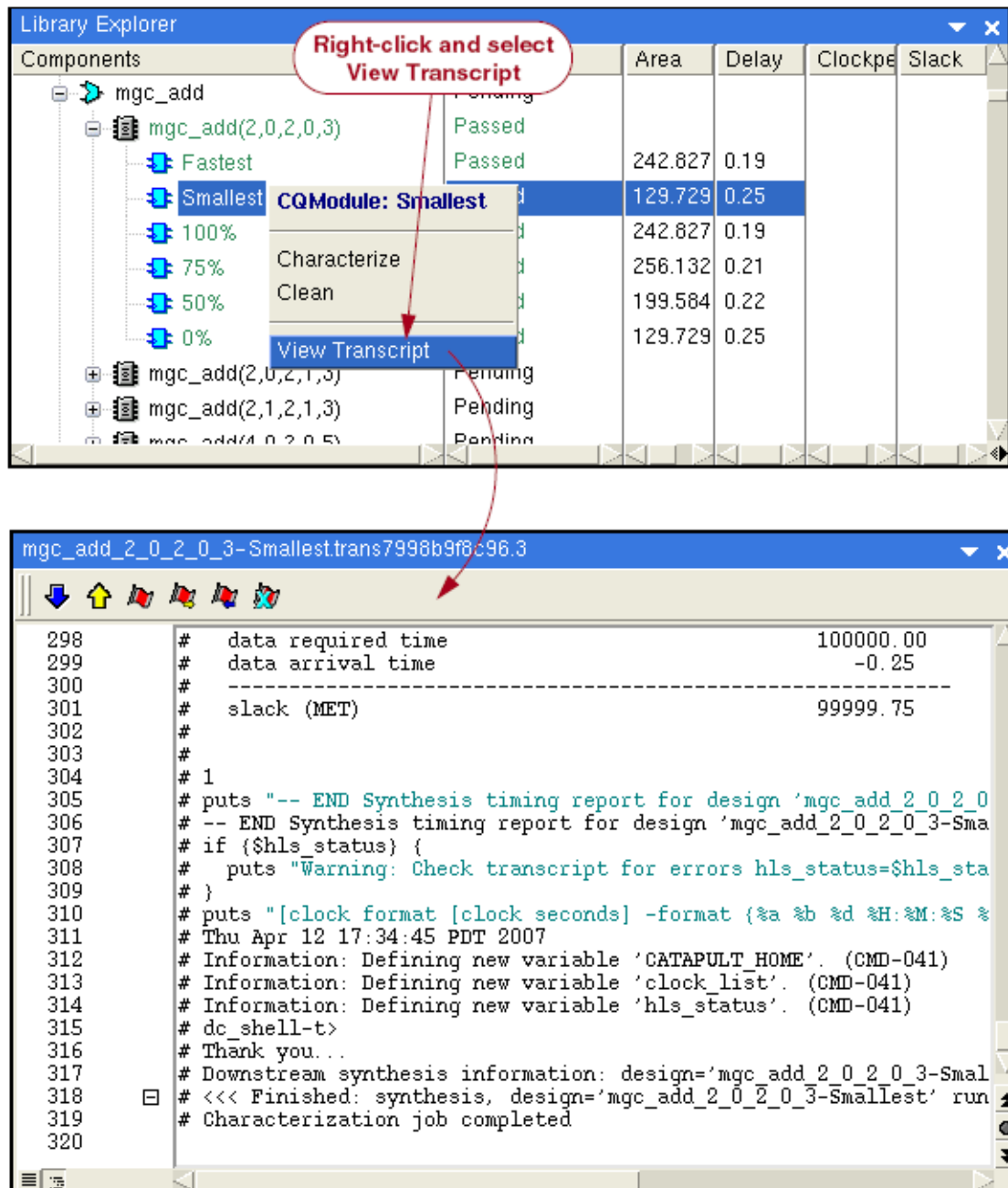


Illustration 19: Viewing the Characterization Transcript

2.5.7. Reducing Characterization Time

The characterization process can be very time consuming when a large number of QMODs are synthesized for each MOD. In general, the more data points (QMODs) that are measured, the greater the accuracy of the estimated (interpolated) area and timing data for the non-measured QMODs. The Library Builder's rapid characterization option allows you trade-off some amount of accuracy for faster characterization run times.

The rapid characterization method only measures the minimum number of QMODs necessary to achieve a user-specified error threshold (MaxError). The rapid process is iterative. It starts with the absolute minimum number of QMODs needed to construct an interpolation equation that defines the curve shape for the MOD. If after characterizing the initial set of QMODs the interpolation error exceeds the MaxError threshold, another QMOD is added and characterized in order to improve the interpolation accuracy. Additional QMODs are added until the MaxError threshold is achieved or the maximum number of QMODs have been characterized.

Creating a Rapid Characterization Library

The *rapid* characterization option must be set when a library is created. The newly created library contains fewer QMODs than a standard library. From the GUI, enable the **Minimize characterization points** option on the Settings tab of the Library Creation dialog box. From the command line, use -rapid switch. For example:

```
flow run /<rtl_synth_tool>/library add <lib_template> ...  
                                           -rapid <true_of_false> ...
```

Setting the MaxError Threshold

MaxError threshold values are optional and are not defined in the initial rapid library. If after characterizing a MOD you determine that it needs a threshold defined, edit the interpolation expression(s) for the MOD's characterization properties and add the *maxerror()* expression.

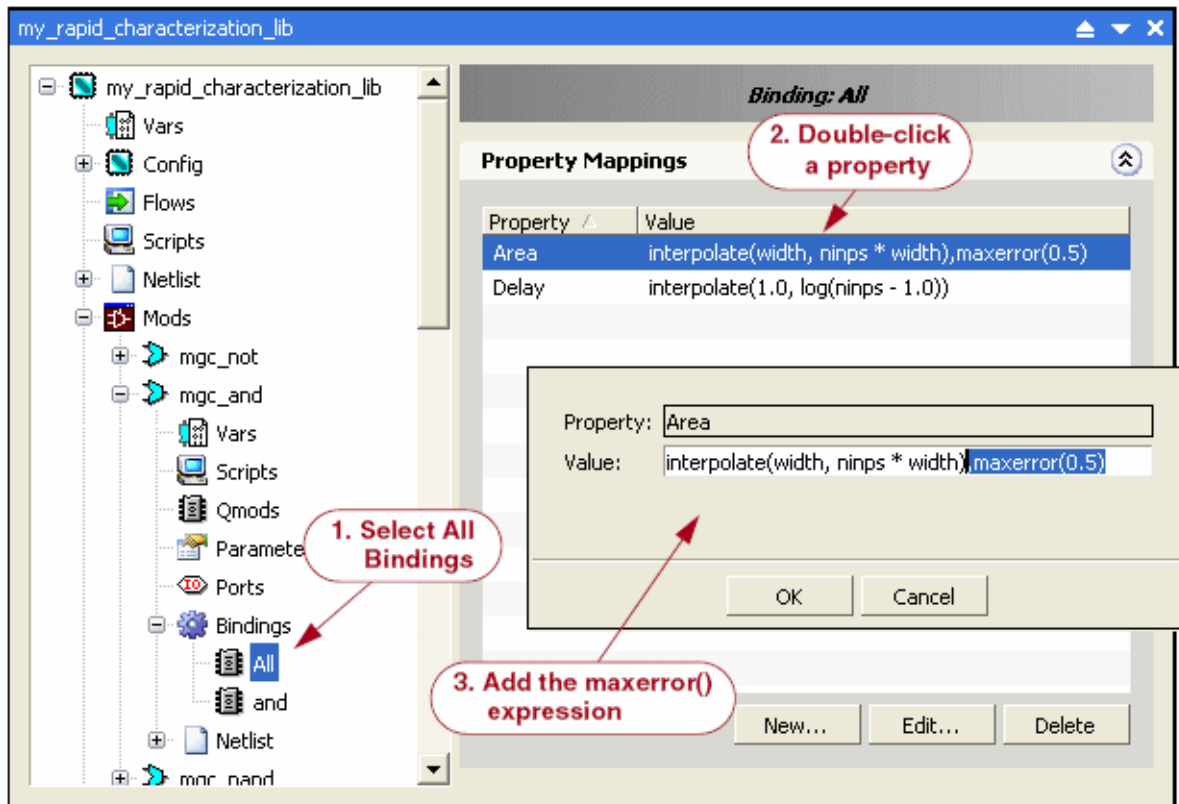


Illustration 20: Defining MaxError for Module Property Mappings

The command syntax for editing an *All Bindings* property is as follows:

```
library set /LIBS/<lib_name>/MODS/<mod_name>/BINDINGS/all/PROPERTY_MAPPING --
-Area {<interpolation_expression>}
```

For example:

```
library set /LIBS/my_catc_lib/MODS/mgc_and/BINDINGS/all/PROPERTY_MAPPING --
-Area {interpolate(width, ninps * width),maxerror(0.5)}
```

2.6. Saving Libraries

Library edits are not saved until you explicitly save the library to disk with one of the following options:

1. Select **Save All Libraries As** from the *Library Tasks* window or the *File* pulldown menu. This command saves all open libraries together as a single file. The default file name is `<synthesis_lib_prefix>_<RTL_synth_tool>.lib`.
2. From within the *Library Explorer* window, right-click on a library and select the **Save As...** menu item.

The default file name is the name of the selected library.

In both cases a file system browser is provided for you to navigate to the location where you want the file saved. Place the library in the Catapult search path to make available to Catapult sessions. If you save it in an alternate location, you must add that path to the Catapult library search path (Use **Tools > Set Options > Component Libraries** pulldown menu within Catapult).

2.7. Troubleshooting Library Failures

If the library characterization fails, Library Builder displays a message about the failure. If this occurs, you can launch Catapult to further investigate the failure. Use Catapult to modify the component and synthesize it. The area and delay data obtained in Catapult can be copied into your library in Library Builder.

NOTE: *This option is not available if you are using Library Farm. If you are experiencing farm failures but can't identify the issue, check that the necessary environment variables are getting passed to the farm machine.*

Procedure:

1. If the *Remove Project Directories for Local Tasks* option is enabled, disable it and run the *Characterize* command on the object again.

Library Builder always creates a Catapult project directory for each characterization task. By default, the project directories are deleted when the task is finished. Disabling this option will preserve the directory.

To disable the option in the General Options pane (Tools > Set Options...), or use the following command:

```
options set General RemoveProjectDirectories false
# false
```

2. Launch Catapult by right-clicking on the failed library object in the Library Explorer window and select **Open Catapult Project** from the popup menu as shown in Illustration 22. This will open a Catapult session, load the failed module into the project and generate RTL.
3. Modify the component as needed to fix the problem and generate RTL.

4. Double-click on the *Synthesize rtl.vhdl* makefile to launch Design Compiler. Alternatively, right-click on *Synthesize rtl.vhdl* and select the *Launch DesignCompiler* command from the popup menu.
5. After obtaining the area and delay data in Catapult, use the Library Explorer window to manually enter the data for the failed object. Double-click on a data field to edit, such as **Area**, **Delay** or **Clockperiod** and enter the value. If a valid value is entered in the field, the status for the component will be changed to Passed.

Components	Status	Area	Delay	Clockperiod
mgc_add	Failed Generate			
mgc_add(2,0,2,0,3)	Failed Generate			
Fastest	Failed Generate			
Smallest	Failed Generate			
100%	Passed	242.827	0.19	0.19
75%	Passed	256.133	0.21	0.205
50%	Passed	199.584	0.22	0.22
0%	Passed	129.73	0.25	0.25
mgc_add(2,0,2,1,3)	Passed			

Illustration 21: Entering New Area and Timing Values

The following illustration shows how to open a QMOD that failed to generate results.

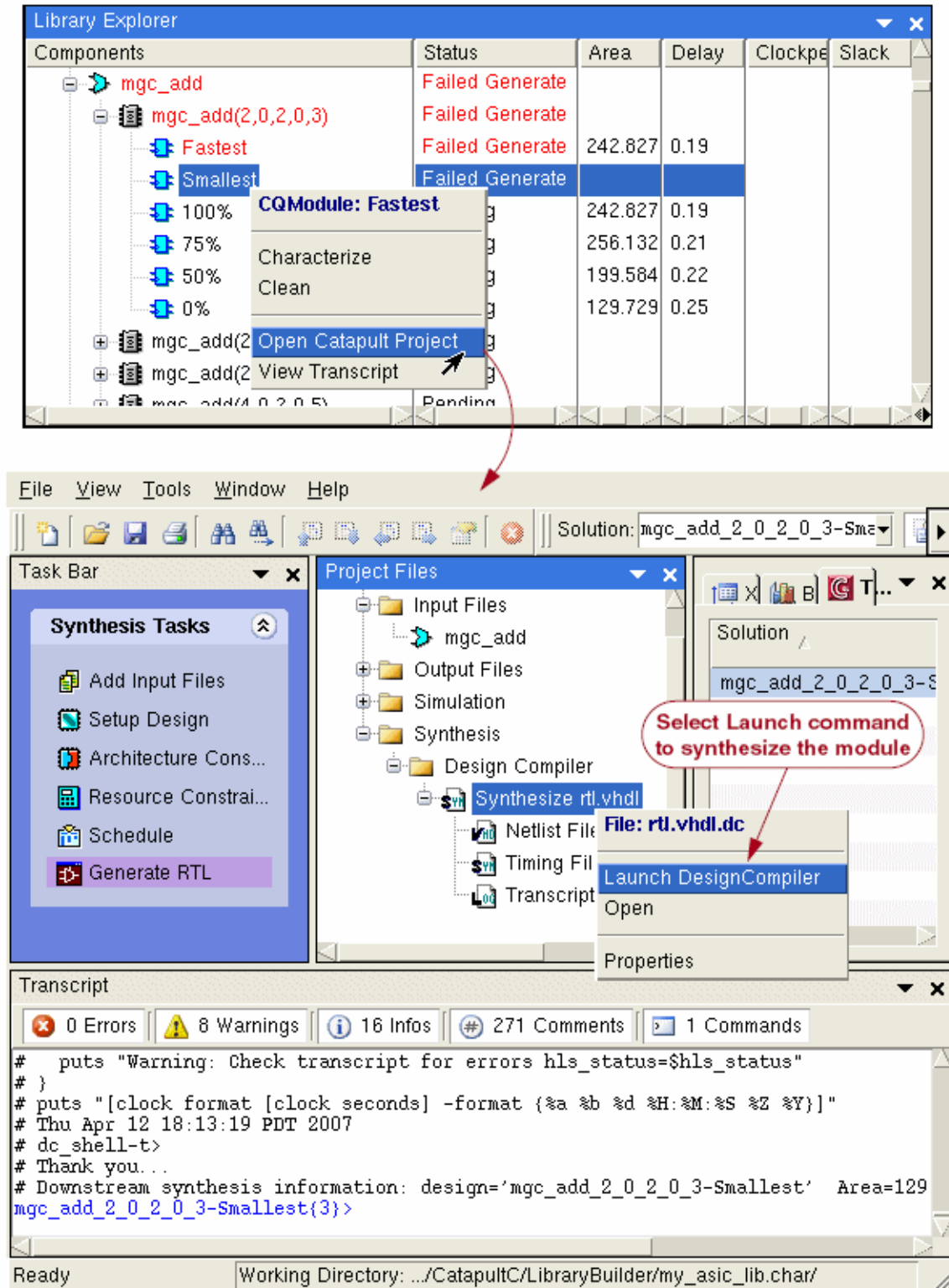


Illustration 22: Opening Failed Object in Catapult

2.8. Improving the Accuracy of Estimated Characterization Values

Using the measured characterization data obtained from RTL synthesis tools, the Library Builder uses a curve fitting algorithm to estimate intermediate area and delay values. The Library Builder generates a curve fitting equation for each characterization property (e.g. Area, Delay and MinClkPrd) in a MOD based on the *curve shape* expression for that property. The library templates define default characterization curves for each MOD property. It may be possible to achieve more accurate estimates by modifying default curve shapes. That is done by editing the property values in the library.

Other library settings also affect the number of estimated data points. The set of valid combinations of parameters is limited by parameter value ranges and the *Always* property, if set. The set of characterized (measured) combinations of parameters is limited by the specified characterization range (*dataset* parameter) and the *Always* property, if set.

2.8.1. Specifying Characterization Curve Shape

The characterization curve is specified through the multi-argument `interpolate()` and `expand()` expressions. The shape is defined as an expression of terms with the characterization parameters as variables. The `interpolate()` expression specifies the shape of a single curve to fit the full set of *measured* data points (obtained during characterization). The `expand()` expression can be used to refine the `interpolate()` curve by segmenting it and fitting curves to each segment.

Illustration 23 shows an example of how the `interpolate()` and `expand()` expressions are used together for the Area and Delay properties of the `mgc_xor` MOD.

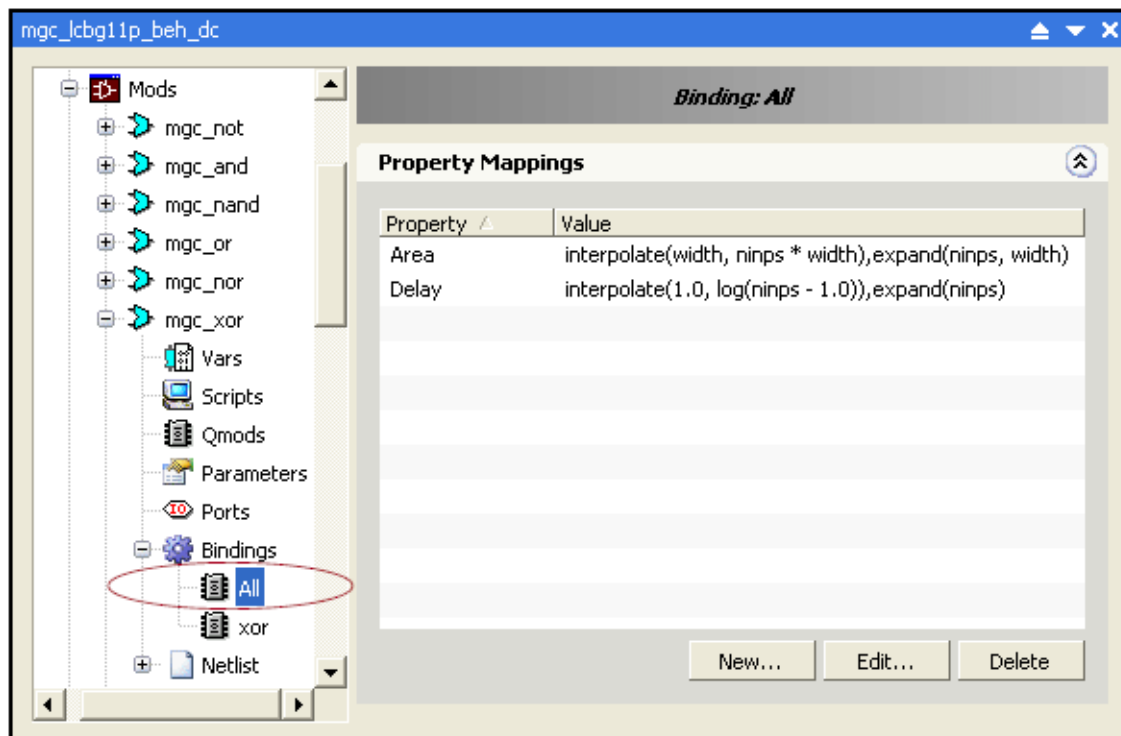


Illustration 23: Characterization Curve Shape Specifications

Syntax for the interpolate() expression:

```
interpolate(<term1>, ...)
```

where <term> is a characterization parameter for the given MOD or an expression.

Syntax for the expand() expression:

```
expand(<param>, ...)
```

where <param> is a characterization parameter for the given MOD. The expand() expression will have different functionality depending on the kind of a parameter. Expanding a binary parameter will yield two curves, one for each value of the binary parameter.

For non-binary parameters, Library Builder will segment the interpolate() curve at its measured characterization points and fit curves at each segment, using at least n number of points to uniquely define the particular type of a curve ($n=2$ for straight line, $n=3$ for a parabola, and so on). The following example demonstrates how expand() can improve the accuracy of the interpolate() curve.

Illustration 24 shows a plot of the interpolate() curve for the Area property of the mgc_xor MOD. The interpolate() expression is specified by two terms:

```
interpolate(width, width*ninputs)
```

where *ninputs* is the number of inputs of a single XOR gate in the module and *width* is the number of independent outputs (XOR gates in parallel). In the plot, the red and yellow circles are the measured characterization points, and the small blue circles are the estimated points.

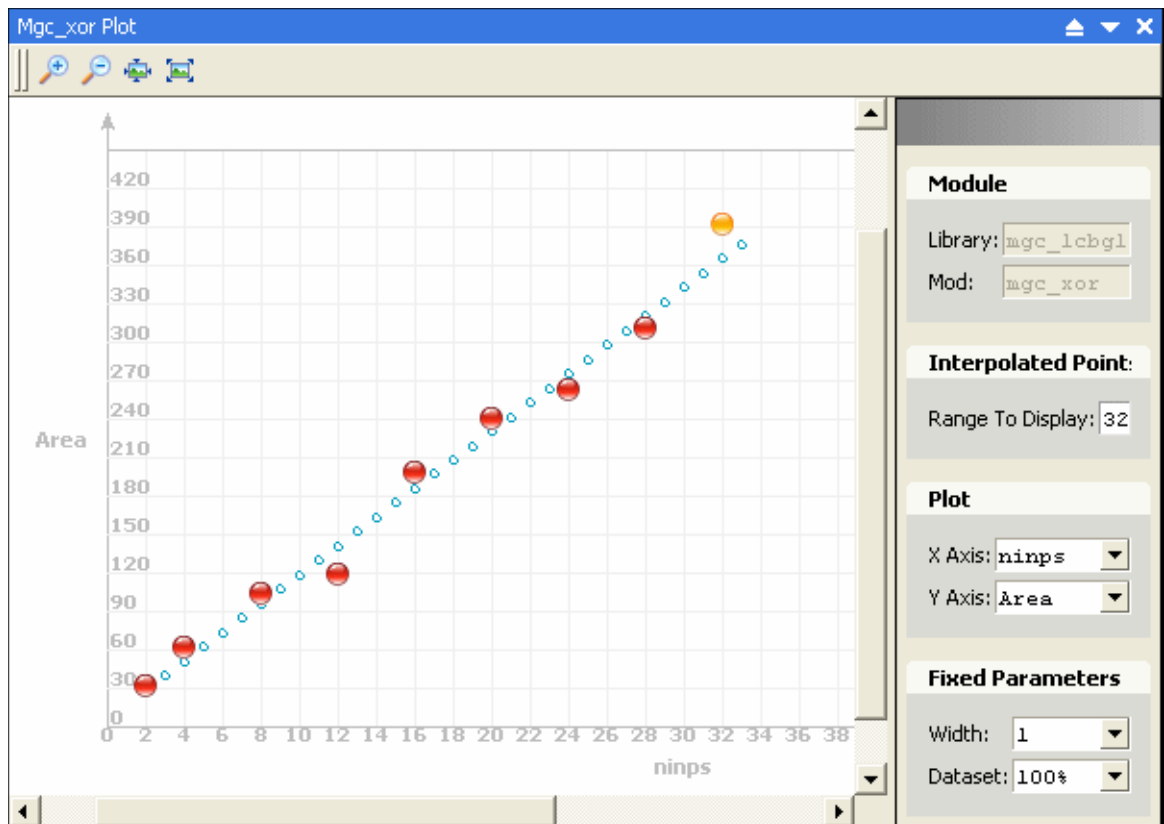


Illustration 24: Interpolate() Characterization Curve

The plot in Illustration 25 shows that better correlation between measured and estimated data values is achieved by including the `expand()` expression. The `interpolate()` expression is unchanged. The curve specification for this plot is:

```
interpolate(width, ninps * width),expand(ninps, width)
```

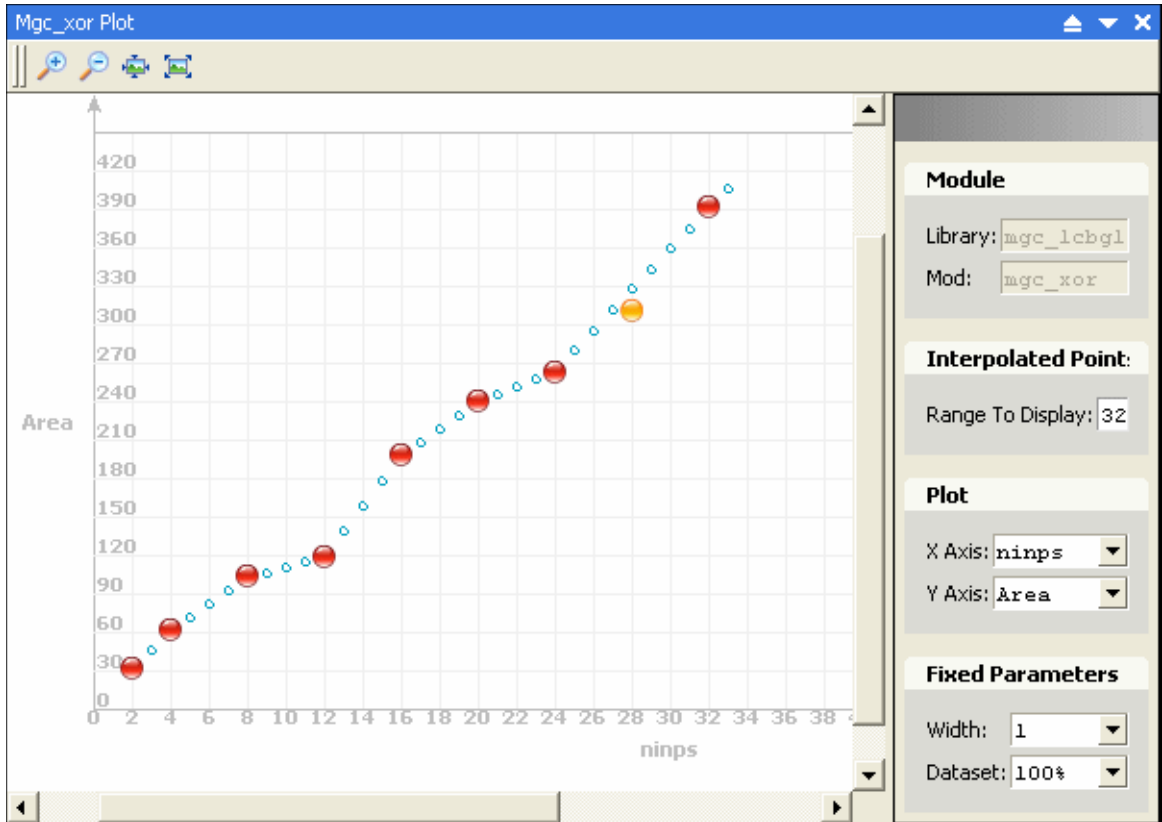


Illustration 25: Interpolate() and Expand() Characterization Curve

2.8.2. Editing Characterization Properties

As shown Illustration 26, to edit a module property with the Library Editor GUI, select the *All* binding of a module to access its properties. To add a new characterization property, click the New button. Otherwise select an existing property and click the Edit button. Fill in the dialog box and click the OK button.

From the command line, use either the [library add](#) or [library set](#) commands to create or modify, respectively, library properties. The command syntax is the same for both commands:

```
library add|set /LIBS/<lname>/MODS/<mname>/BINDINGS/all/PROPERTY_MAPPING --\
-<property_name> {<expression>}
```

Example:

```
library set /LIBS/mgc_lcbgl1p_beh_dc/MODS/mgc_xor/BINDINGS/all/PROPERTY_MAPPING
--\
-Area {interpolate(width, ninps * width),expand(ninps, width)}
```

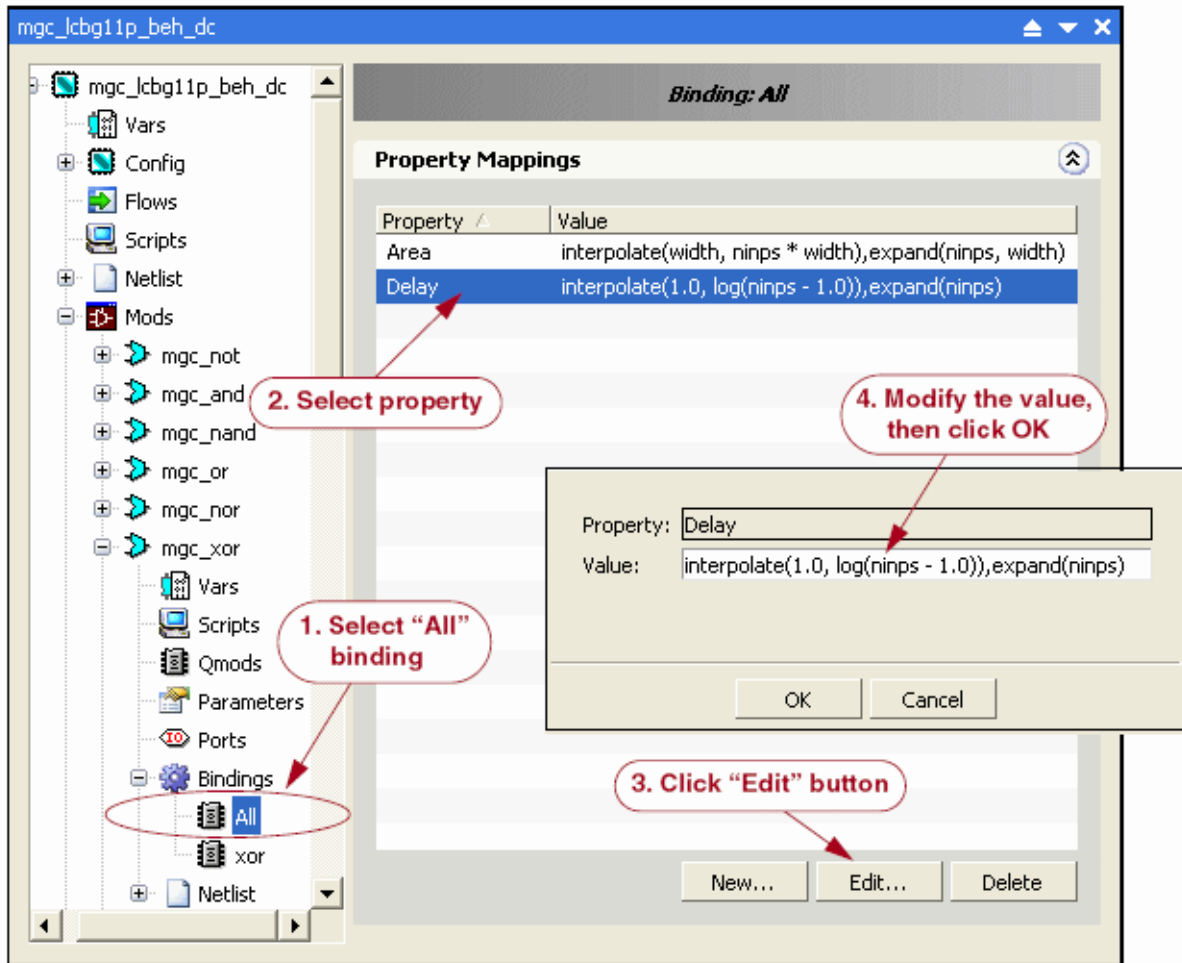


Illustration 26: Editing Characterization Properties

2.9. Using the Library Farm

Library Builder contains a Library Farm tool that can be used distribute library characterization tasks to host computers on your network, thereby speeding up library characterization processing by allowing the tasks to run in parallel.

NOTE: You should verify that the .cshrc sources all scripts required to enable the RTL synthesis tool and Catapult, so rsh is able to just start running the tools. Make sure to check the number of RTL synthesis tool licenses available or the Farm may consume more than are available.

Characterization reports do not include or reflect failures due to license issues. If you are unsure why a Library Farm job is failing, try checking to ensure that the necessary license environment variables are being passed to the farm machine.

To set up the Library Farm tool:

1. **Enable the Farm:**

The factory default settings have the Farm disabled and the Farm window hidden. To make the window visible, select the **View > Farm** menu item. To enable the Farm, right-click in the Farm window and choose Enable Farm from the popup menu. If you want the Farm to be enabled by default, modify the settings in the [Farm Options](#) (**Tools > Set Options.... > Farm**)

2. **Configure Library Builder** for your farm. Most farms use one of the following two load balancing software types. Examples on how to set up jobs on your farm (e.g. OS, RAM requirements, and licensing) can be found in the following options. These settings should be the same as your RTL Synthesis flow.
 - [Load Sharing Facility \(LSF\) software](#)
 - [Sun Grid Engine \(SGE\)](#)

After you have configured the farm, you can add hosts.

3. Setting Up Library Farm Hosts will explain how to set up a host to run jobs on your farm.
4. **Test the farm configuration.**
 - a. Pick one QMOD (e.g. mgc_not) and run that on your farm to make sure the RTL synthesis tool runs, a transcript is produced, and the Library Explorer table contains numbers from characterization. If the Library Explorer table is empty, right-click on the QMOD and select View Transcript from the menu.
 - b. After one QMOD runs successfully, run one MOD on the farm. If the MOD runs successfully, you can run your library.

2.9.1. Setting Up Library Farm Hosts

The Library Farm window displays the list of hosts and the status of each task, such as idle, passed or failed. The initial Farm window is empty.

1. To set up the first host, right-click in the blank area on the Farm window and choose **Add a Host** from the pop-up menu to open the Add a Host dialog box, as shown in the illustration below.

The host name localhost is a reserved name for the local machine. Jobs sent to localhost are run on the local machine without any network requirements. This can be useful for multiprocessor machines where you would like to run additional characterization tasks.

2. Enter the name of the host computer and specify the number of tasks for that host.

The number of tasks you can set for each host is limited by the number of RTL synthesis tool licenses available.

3. Apply the settings by clicking **OK**.

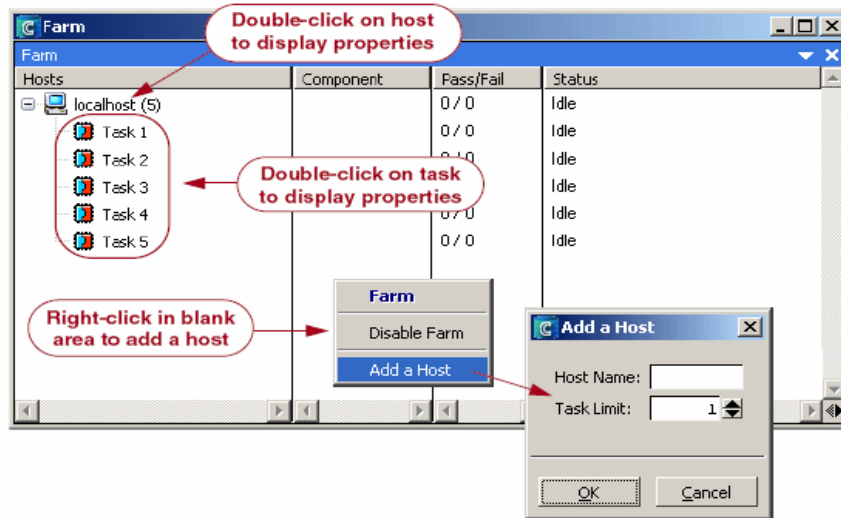


Illustration 27: Library Farm Window

Right-click on a host name to open the Host menu. This menu allows you to add another task, remove the host, or see properties associated with the host.

- If you select **Add another Task**, a new task is added to the list in the Farm window. The pass/fail values are zero and the status is set to idle.
- If you select **Remove**, the host is removed from the Farm list.
- If you select **Properties**, the Farm Properties dialog box opens, with which you can change the number of tasks for that host. Use the scroll-arrows, or type the desired number of tasks and click **OK** or **Cancel**.

Task Menu: Right-click on a task to open its pop-up menu. The menu allows you to remove the selected task or view its properties.

For more information on setting up Library Farm hosts, see the [Farm Window](#) section.

Configuring Library Farm to Use the Load Sharing Facility (LSF) software

To configure Library Farm to work with LSF, modify the **Remote Shell Command** setting on the Farm options page and set to an appropriate LSF command line. The format of the command line will include LSF command switches and Catapult internal variables as arguments. For information about these internal variables, refer to the [Farm Options](#) help. You can also set the Remote Shell Command option by using the [options set](#) command as shown in the examples below.

NOTE: You must add placeholder “hosts” to the Library Farm in order to specify the number of jobs to submit to LSF. Any name may be used, except “localhost,” which is a reserved name. For example, you might add a host named “LSF” and a task limit of 5. That would imply that five LSF jobs may be submitted concurrently. You may adjust the number of tasks while Library Builder is characterizing.

Example 1: Specify LSF *bsub* command line to run on a local machine

Set the following option in Catapult:

```
options set Farm RshCommand {bsub -o %OUTPUTFILE% -q long  
-R "select[rh30_32b==1]" -L /bin/csh -K %COMMAND% -file %COMMANDFILE%}
```

- The **-o %OUTPUTFILE%** option specifies that upon completion of the job the standard output should be placed in the file named by **%OUTPUTFILE%** on the local host.
- The **-q long** option specifies the queue name for the job. The queue is named **long** in this example.
- The **-R "select[rh30_32b==1]"** option specifies the resource requirements of the job.
- The **-env** argument specifies a comma-separated list of job submission environment variables to propagate, this can be useful for ensuring the farm machine has access to licenses. Some common argument options include:
 - “<var_name>=<var_value>” – Specifies a variable name and value to pass to the farm machine. For example:

```
-env "VAR1=val1, VAR2=val2"
```

- “none” – If you specify none with no other variables, no environment variables are passed along while submitting the job.
- “all” – Specify this option at the beginning of the argument list to pass along all environment variables, you can also specify specific values after the all option

If **-env** and **-L** conflict, **-L** takes priority. For full details about this and other arguments, consult the official LSF documentation.

- The **-L /bin/csh** option starts a login shell and configures the environment with the user’s login scripts. This only matters if you are launching between different OS platforms. The **-L <arg>** option may be omitted if you are using the same OS for the submitted job.
- The **-K** option specifies that the **bsub** command should not return until the job is completed.
- The **%COMMAND%** option specifies the Catapult command to be invoked

NOTE: When the Library Builder is configured to use dedicated licenses for characterization tasks instead of Catapult licenses, you must use the **%COMMAND%** variable in the Remote Shell Command field. Do not enter a literal invocation command line. The variable automatically supplies special command line flags that are required for the new license.

- The **-file %COMMANDFILE%** option is the Catapult command line option to supply a script file.

Example 2: Specify the LSF command to run from remote LSF *gate* machine:

Set the following option in Catapult:

```
options set Farm RshCommand {rsh %HOSTNAME% "cd %CWD%; bsub -o %OUTPUTFILE%  
-q long -R \"select[rh30_32b==1]\" -L /bin/csh -K %COMMAND% -file %COMMAND-  
FILE%"} }
```

In this example the entire **bsub** command is passed as an argument to an **rsh** command. The **rsh** command takes two arguments, the name of the host machine (**%HOSTNAME%**) and the command string to be executed by **rsh**. Notice that the second argument is enclosed in double quotes and any nested quotes within it (such as **-R \"select[rh30_32b==1]\"**) must be escaped with a backslash.

Example 3: Running Library Builder from an xterm on a LSF host

If xterm is already running on a LSF host with some cores already allocated, you can run Library Builder from this xterm so that it appears to be *'local'* to Library Builder. To accomplish this task, set the following options in Catapult:

```
options set Farm/RshCommand %COMMAND%  
options set Farm/EnableFarm true  
utility farm add -name host -count 8
```

Using these options, Library Builder will run the characterization scripts on the xterm which is already running on a LSF host. The host name specified in the *utility farm add* command can be any arbitrary name because the **%HOSTNAME%** is not used (*Farm/RshCommand* only passes the **%COMMAND%**).

Configuring Library Farm to Use the Sun Grid Engine (SGE) software

To configure Library Farm to work with LSF, modify the **Remote Shell Command** setting on the Farm options page and set to an appropriate LSF command line. The format of the command line will include LSF command switches and Catapult internal variables as arguments. For information about these internal variables, refer to the [Farm Options](#) help. You can also set the Remote Shell Command option by using the [options set](#) command as shown in the examples below.

Example 1: Specify SGE command line to run on a local machine

Set the following option in Catapult:

```
options set Farm/RshCommand {grid qsh -o %OUTPUTFILE% -q long  
-R "select[rh30_32b==1]" -L /bin/csh %COMMAND% -f %COMMANDFILE%}
```

- The **-o %OUTPUTFILE%** option specifies that upon completion of the job the standard output should be placed in the file named by **%OUTPUTFILE%** on the local host.
- The **-q long** option specifies the queue name for the job. The queue is named **long** in this example.

- The **-R "select[rh30_32b==1]"** option specifies the resource requirements of the job.
- The **-L /bin/csh** option starts a login shell and configures the environment with the user's login scripts. This only matters if you are launching between different OS platforms. The **-L <arg>** option may be omitted if you are using the same OS for the submitted job.
- The **%COMMAND%** option specifies the Catapult command to be invoked

NOTE: When the Library Builder is configured to use dedicated licenses for characterization tasks instead of Catapult licenses, you must use the **%COMMAND%** variable in the Remote Shell Command field. Do not enter a literal invocation command line. The variable automatically supplies special command line flags that are required for the new license.

- The **-file %COMMANDFILE%** option is the Catapult command line option to supply a script file.

Example 2: Specify the SGE command to run from remote *gate* machine:

Set the following option in Catapult:

```
options set Farm RshCommand {grid qrsh %HOSTNAME% "cd %CWD%; bsub -o %OUTPUT-  
FILE%  
-q long -R \"select[rh30_32b==1]\" -L /bin/csh -K %COMMAND% -file %COMMAND-  
FILE%"} }
```

In this example the entire **bsub** command is passed as an argument to an **rsh** command. The **rsh** command takes two arguments, the name of the host machine (**%HOSTNAME%**) and the command string to be executed by **rsh**. Notice that the second argument is enclosed in double quotes and any nested quotes within it (such as **-R \"select[rh30_32b==1]\"**) must be escaped with a backslash.

Example 3: Running Library Builder from an xterm on a SGE host

If xterm is already running on a LSF host with some cores already allocated, you can run Library Builder from this xterm so that it appears to be *'local'* to Library Builder. To accomplish this task, set the following options in Catapult:

```
options set Farm/RshCommand %COMMAND%  
options set Farm/EnableFarm true  
utility farm add -name host -count 8
```

Using these options, Library Builder will run the characterization scripts on the xterm which is already running on a LSF host. The host name specified in the *utility farm add* command can be any arbitrary name because the **%HOSTNAME%** is not used (*Farm/RshCommand* only passes the **%COMMAND%**).

Chapter 3: Editing Libraries

After Library Builder has created the library, you can edit the library to provide additional information in order to build a valid library. This information consists of library data such as latency, throughput behavior, and parameter ranges. To edit the library, right-click on the library in the **Library Explorer** tab and select **edit**.

Each library consists of a set of Variables that affect the entire library and a set of Modules (MODs). Catapult determines the area and delay for operators using the modules in the library. Catapult also uses information in each module to determine how to instantiate the operator within the RTL netlist. The following sections describe how to view and edit each section of the library:

- [Updating Catapult Libraries with New Components](#) add new components from an existing base library template.
- [Editing Library Variables](#) describes how to edit variables in the GUI and provides a list of all variables in a library.
- [Editing Module Parameters](#) define the list of parameters on the Operator or interface.
- [Editing Module Ports](#) define the inputs and outputs to the operator or interface.
- [Editing Module Bindings](#) map the parameters on the operator to the parameters on the RTL netlist module.
- [Editing Module Pin Associations](#) map the pins on the module to the ports on the operator.
- [Editing Module Property Mappings](#) map the properties (attributes) on the module to the operator.

3.1. Updating Catapult Libraries with New Components

When a base ASIC library is created in Library Builder, the set of components implemented in the library is derived from the ASIC library template shipped with the current version of Catapult. Over time, new base library components made be added to the ASIC library template as part of newer Catapult releases. Rather than re-create an older base library using the new template, Library Builder allows you to refresh an existing library (retaining its characterization data) with updated components from the new ASIC library template. Once the new components are added, you can characterize the newly added components to generate area and delay data without modifying the existing base library components.

Library Builder combines these new components into groups of similar components. These groups, when added, will expand the set of components in your existing library. The following list briefly describes the currently available component groups (and the corresponding switches for the *library refresh* command):

- **Advanced Components.** (*-include_advanced_components <true|false>*) These components allow Catapult Ultra to perform additional optimizations. Refer to the [ENABLE_ADVANCED_COMPONENTS](#) directive.

- **DesignWare Components.** (*-dware_support <append|none>*). These components provide DesignWare implementations for some Catapult operators. The Catapult-generated netlist will include DesignWare instances of these components. The SCVerify flow will automatically compile the simulation models from your Synopsys install tree. These components are blackboxes in the LowPower flow through PowerPro.

To use refresh your library, follow these steps:

1. Invoke Library Builder

```
catapult -product lb
```

2. Load existing Catapult Library into Library Builder

```
# library load <Catapult_library_path>
library load $env(HLS_LIB)/tech7nm/tech7nm_beh.lib
```

3. Check for new components using the *library refresh* command. If you loaded multiple libraries into Library Builder, you must specify the *-library* switch.

NOTE: This step only reports possible new components and does not modify the existing library.

```
library refresh -include_advanced_components 1 -dware_support append
# Processing library 'tech7nm_beh'...
# Checking for new library components...
# Library template 'mgc_tmpl_beh_dc' has these components available to be
imported into this library: 'mgc_add_msb ccs_dw_div ccs_dw_div_pipe ccs_d-
w_lp_piped_div'
```

If the default ASIC library template contains any components are not in the current library, Library Builder will list the names of the new components in the transcript. If no new components are available, Library Builder will not provide any information beyond the “Processing” and “Checking” info statements.

4. If needed, add the new group(s) of components using the *library refresh -update 1* command.

```
library refresh -include_advanced_components 1 -update 1
# Info: Processing library 'tech7nm_beh'...
# Info: Checking for new library components...
# Info: Adding module 'mgc_add_msb' to library tech7nm_beh
```

This step will add the new components to the existing library. You can delete any of the new components by selecting the component in the Library Explorer and picking “Delete” from the popup menu. For example, you may want the new DesignWare components but do not need the pipelined versions.

5. Characterize the Library

After the components have been added to the base technology library, you can characterize the new components to add area and delay data to the Catapult library. Library Builder will only characterize components with missing data.

6. Save the new version of the library with the characterized data of the new components.

The *library refresh* command in Library Builder provides a simple process for updating existing Catapult libraries with the latest components from the ASIC library template.

3.2. Editing Library Variables

To change library variables after a library is created, you must use the Library Builder as described in the following procedure.

1. Load the library into Library Builder (*catapult -product lb*)
2. Click on the Library Explorer tab. The Library Explorer window displays.
3. Double-click on the library to edit. The Library edit tab opens to display the settings and components in the library.
4. Click **Vars** as shown in the following illustration. The defined variables are now displayed for the library as shown in the following illustration:

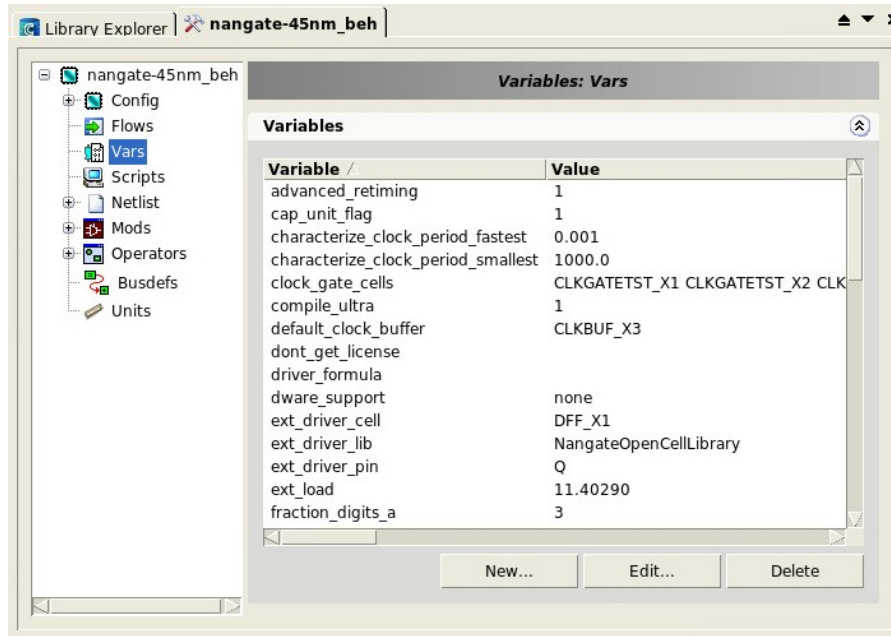


Illustration 28: Viewing and Editing Library Variables

5. Depending on the modifications, use the New, Edit, and Delete buttons to modify the library variables as needed.
6. Save the changes to the library: click the Library Explorer tab, right-click the library that you would like

to save and select Save from the menu.

The following table describes each of the library variables. For obsolete library variables, the table describes the variables to be used instead. Unless noted in the table, you can infer the following information from each variable:

- Library Builder stores variables under `/LIBS/<lib_name>/VARS`.
- For each variable, the following table denotes the corresponding Library Wizard field name and [flow run](#) switch. Library Builder uses the values from the Library Wizard to generate the `flow run` command.

```
flow run <RTLSynTool>library add base -selected <bool> -rapid false <switches>
```

- To ensure good correlation between the HLS and RTL synthesis results, most variable values should match the corresponding command in your RTL synthesis script.

VAR Name	Description
advanced_retiming	Type: <i>boolean</i> . Default: <i>true</i> . RTL Tool: <i>DC</i> Wizard Name: <i>Advanced Retiming</i> Flow Run Switch: <code>-set_advanced_retiming <bool></code> If true the optimize_registers is used for retiming, the balance_registers is used otherwise.
black_box	Type: <i>boolean</i> . Default: <i>-</i> . RTL Tool: <i>All</i> Wizard Name: <i>None</i> Flow Run Switch: <i>None</i> If set on a library component, then downstream flows can detect when the component should be blackboxed (eg. DesignWare components in the LowPower flow).
cap_factor	Type: <i>float</i> . Default: <i>1</i> . RTL Tool: <i>All</i> Wizard Name: <i>none</i> Flow Run Switch: <i>None</i> Must be equal to capacitance unit factor in technology library. Specifies the capacitance unit factor.
cap_table	Type: <i>string</i> . Default: <i>{}</i> RTL Tool: <i>RC</i> Wizard Name: <i>Capacitance Table Files</i> Flow Run Switch: <code>-cap_table <string></code> A space separated list of Capacitance Tables files. Catapult passes this value to the " <code>set_attr cap_table_file <list></code> " command in the generated script file for RC.
cap_unit	Type: <i>pick list (nf, pf, ff)</i> . Default: <i>ff</i> RTL Tool: <i>All</i> Wizard Name: <i>Capacitance Unit</i> Flow Run Switch: <code>-cap_unit <ns pf ff></code> Allowed values are nf, pf, and ff. Must be equal to capacitance unit in technology library. In conjunction with cap_factor specifies the capacitance unit.
characterize_clock_period_fastest	Type: <i>double</i> . Default: <i>"0.01"</i> (nS). RTL Tool: <i>All</i> Wizard Name: <i>Clock Period Fastest</i> Flow Run Switch: <code>-characterize_clock_period_fastest <double></code> This value specifies the clock speed setting for RTL Synthesis when characterizing the fastest clock period (fastest implementation with largest area).
characterize_clock_period_percents	Type: <i>string</i> . Default: <i>{100% 75% 50% 0%}</i> RTL Tool: <i>All</i> Wizard Name: <i>Clock Period Percents</i> Flow Run Switch: <code>-characterize_clock_period_percents <string></code> This field specifies a set of points within the range of clock speeds bounded by the fastest and smallest settings. The points are specified as percentages of the range. The default set consists of four points: 100%, 75%, 50% and 0%, where 100 = fastest and 0 = smallest. Arithmetic operators (eg. mgc_add, mgc_mul) may have more points to improve correlation.
characterize_clock_period_smallest	Type: <i>double</i> . Default: <i>1000.0</i> (ns). RTL Tool: <i>All</i> Wizard Name: <i>Clock Period Smallest</i> Flow Run Switch: <code>-characterize_clock_period_smallest <double></code> This value specifies the clock speed setting for RTL Synthesis when

VAR Name	Description
	characterizing the slowest clock period (smallest area).
chip_util	Type: integer Default: 40.0 RTL Tool: Oasys, Aprisa RTL Wizard Name: Chip Utilization Flow Run Switch: -chip_util <int> Sets the default chip utilization for the initial floorplan. Based on this value and the area estimate for the design, the RTL synthesis tool creates the correspondingly sized block in the floorplan.
clock_gate_cells	Type: space-separated <i>list</i>. Default: {}. RTL Tool: All Wizard Name: Clock Gate Cells Flow Run Switch: -clock_gate_cells <string> If this optional variable includes at least one cell and the Clock Gate Min Width flow option is greater than 0, then Oasys will synthesize the RTL netlist generated by Catapult with clock gates (adds -gate_clock switch to synthesize command).
compile_to_placed	Type: boolean. Default: false. RTL Tool: RC Wizard Name: Compile Mode to placed Flow Run Switch: -set_compile_to_placed <bool> If set to true, Catapult (and Library Builder) will pass the -to_placed switch to RC's compile command.
compile_ultra	Type: boolean. Default: false. RTL Tool: DC Wizard Name: Set Compile Mode to Ultra Flow Run Switch: -set_compile_ultra <bool> Specifies whether to run Design Compiler in ultra mode. Since the characterized operators are generally small, using DC Ultra tends to increase the run time of characterization without significant effect on the area or timing estimates. During characterization, you should set this variable to false. For your Design Compiler runs, you should continue to use this option during RTL synthesis.
dont_use_cells	Type: string. Default: {} RTL Tool: All Wizard Name: Excluded Cells Flow Run Switch: -set_dont_use_cells <string> Specifies a list of technology cells that will not be used during RTL synthesis. This optional list should contain <library name> <cell name> pairs. This value is passed to the dont_use command.
dont_get_license	Type: string. RTL Tool: DC Wizard Name: Design License not to use Flow Run Switch: -set_dont_get_license <string> Specifies a list of DesignWare licenses that the DC tool is not allowed to use. The list items are space separated. The list is assigned to the DC variable <i>synlib_dont_get_license</i> in the generated DC script. This value will be the default setting for users of the Catapult library.
default_clock_buffer	Type: string. Default: "". RTL Tool: All Wizard Name: Default Clock Buffer Flow Run Switch: -set_default_clock_buffer <string> This variable defines the default value for the <i>clock_tree_buffer</i> option in the

VAR Name	Description
	Low-Power flow. In Catapult's Low-Power Option form, you can override the buffer name in the library.
disable_retiming	Type: <i>boolean</i> . Default: <code>""</code> . RTL Tool: <i>Oasys, RTL Compiler</i> Wizard Name: <i>Disable Retiming</i> Flow Run Switch: <code>-set_disable_retiming <bool></code> If set to true, 'retiming' will not be performed during characterization
dware_support	Type: <i>picklist (none, append, replace)</i> . Default: <i>None</i> . RTL Tool: <i>DC, Oasys</i> Wizard Name: <i>Include DesignWare Modules</i> Flow Run Switch: <code>-dware_support <none append replace></code> This variable denotes whether to use DesignWare implementations for modules in the Catapult library. By default, no modules use DesignWare components. - If you specify <i>“Replace”</i>, Library Builder will implement <i>mgc_div</i> and <i>mgc_mul_pipe</i> modules with DW components. - If you specify <i>“append”</i>, Library Builder will implement <i>mgc_div</i> and <i>mgc_mul_pipe</i> modules with DW components and include DW Floating Point components.
ext_driver_lib	Type: <i>string</i> . Default: <code>""</code> . RTL Tool: <i>All</i> Wizard Name: <i>External Driver Cell Library</i> Flow Run Switch: <code>-set_ext_driver_lib <string></code> Driver cell is used as a driver for all input pins. This field specifies the library in which the driver cell is found. If left blank no driver cell is used.
ext_driver_cell	Type: <i>string</i> . Default: <code>""</code> . RTL Tool: <i>All</i> Wizard Name: <i>External Driver Cell</i> Flow Run Switch: <code>-set_ext_driver_cell <string></code> Driver cell is used as a driver for all input pins. This field specifies the cell name. If left blank no driver cell is used. Library Builder attempts to select the buffer with an area in the 66th percentile of all buffers in the library. If you are specifying a driver cell, then you must specify the library, cell and pin or else Library Builder will not use a driver cell.
ext_driver_pin	Type: <i>string</i> . Default: <code>""</code> . RTL Tool: <i>All</i> Wizard Name: <i>External Driver Cell Pin</i> Flow Run Switch: <code>-set_ext_driver_pin <string></code> Driver cell is used as a driver for all input pins. This field specifies the name of the output pin of the driver cell. If left blank no driver cell is used. If you are specifying a driver cell, then you must specify the library, cell and pin or else Library Builder will not use a driver cell.
ext_load	Type: <i>float</i> . Default: <i>0</i> . RTL Tool: <i>All</i> Wizard Name: <i>External Capacitive Load</i> Flow Run Switch: <code>-set_ext_load <string></code> Specifies the capacitive load that is added to the each top-level output pin. This value is typically set to match the loading effect of a small buffer from the technology library. The specified value is a number of \$cap_factor \$cap_unit units. Higher value would result in more pessimistic and lower – more optimistic estimations.

VAR Name	Description
fraction_digits	This variable is obsolete . Use fraction_digits_t (for timing values) and fraction_digits_c (for capacitance values) instead.
fraction_digits_a	Type: <i>integer</i> . Default: 3. RTL Tool: Oasys Flow Run Switch: <code>-set_fraction_digits_a <int></code> Wizard Name: <i>Area Report Fractional Digits</i> Range: 0-13. Specifies the number of digits after the decimal point to store for area values in the Catapult library (controls the maximum resolution of the timing values).
fraction_digits_t	Type: <i>integer</i> . Default: 7. RTL Tool: All Flow Run Switch: <code>-fraction_digits_t <int></code> Wizard Name: <i>Timing Report Fractional Digits</i> Range: 0-13. Specifies the number of digits after the decimal point to store for timing values in the Catapult library (controls the maximum resolution of the timing values).
fraction_digits_c	Type: <i>integer</i> . Default: 4. RTL Tool: All Wizard Name: <i>none</i> Flow Run Switch: <code>None</code> Range: 0-13. Specifies the number of digits after the decimal point to store for capacitance values in the Catapult library (controls the resolution of the capacitance values).
get_license	Type: <i>string</i> . Default: <i>none</i> . RTL Tool: DC Wizard Name: <i>None</i> Flow Run Switch: <code>-set_get_license <string></code> Specifies a space-separated list of Synopsys DesignWare license features to be obtained. If the value is set, it is passed directly to the DC command <i>get_license</i> in the generated DC script so the specified license features are checked out at the beginning of synthesis. (They are held until the <i>remove_license</i> command is used or until the program is exited or until the DC shell is closed). Refer to the license key file at your site to determine which licensed features are available. This value will be the default setting for users of the Catapult library.
include_advanced_components	Type: <i>boolean</i> . Default: <i>true</i> RTL Tool: All Wizard Name: <i>Include Advanced Components</i> Flow Run Switch: <code>-include_advanced_components <bool></code> If set to <i>true</i> , Library Builder add MODs for the advanced optimizations in Catapult Ultra. Adding these cells should not significantly impact the overall characterization time and should yield improved QofR in Catapult Ultra. You can also add these components to an existing Catapult library using the --include_advanced_components command in Library Builder.
ieff_iter_max	Type: <i>integer</i> . Default: 9 RTL Tool: DC Wizard Name: <i>None</i> Flow Run Switch: <code>None</code> Specifies the maximum number of times to iterate when the fastest dataset is attempting to converge on the desired Slack target (based on the value of the <i>ieff_wns_target</i> variable). Library Builder only utilizes this variable when <i>ieff_version</i> is set to '2'. This variable is only used during the library

VAR Name	Description
	characterization of the fastest dataset for modules listed in the <i>iter_est_qmods</i> list.
ieff_slack_multiplier	Type: <i>float</i>. Default: <i>0.75</i> RTL Tool: <i>DC</i> Wizard Name: <i>None</i> Flow Run Switch: <i>None</i> Specifies the percentage of the previous slack value to use as the offset for the 'fastest' clock period of the next iteration. Values close to 1 will converge faster to the <i>ieff_wns_target</i> value, but at the risk of missing true fastest implementation. Smaller values push the RTL Synthesis tool to try harder to find that true fastest implementation. The following equation summaries how Library Builder determines the clock period for the next iteration of the fastest dataset. <pre>NewClkPeriod = (WNS*ieff_slack_multiplier) + PrevClkPeriod</pre> Library Builder only uses this value when attempting to converge to the desired Slack target (based on the value of the <i>ieff_wns_target</i> variable) for the fastest dataset of a QMOD. Library Builder only utilizes this variable when <i>ieff_version</i> is set to '2'. This variable is only used during the library characterization of the fastest dataset for modules listed in the <i>iter_est_qmods</i> list.
ieff_wns_target	Type: <i>float</i>. Default: <i>0.000</i> RTL Tool: <i>DC</i> Wizard Name: <i>None</i> Flow Run Switch: <i>None</i> Specifies the target slack value when Library Builder converges to the desired Slack target for the fastest dataset of a QMOD. While 0 may be the desired value, in reality, a small number might be just as good with fewer iterations. Library Builder only utilizes this variable when <i>ieff_version</i> is set to '2'. This variable is only used during the library characterization of the fastest dataset for modules listed in the <i>iter_est_qmods</i> list.
ieff_version	Type: <i>integer</i>. Default: <i>1</i> RTL Tool: <i>DC</i> Wizard Name: <i>Design License(s) to check out</i> Flow Run Switch: <i>None</i> Specifies the version of the smoothing function used during the iterative estimation process. Allowed values are 0, 1 or 2. If the VAR does not exist in the library, Catapult defaults to version '1'. This setting ensures that the same iterative method will be applied for all existing and new QMODs (including those QMODs created via the Catapult-generated <i>adjust_char_library.tcl</i> script)
input_transition	Type: <i>float</i>. Default: <i>unset</i>. RTL Tool: <i>All</i> Wizard Name: <i>Input Transition (Slew)</i> Flow Run Switch: <i>-set_input_transition <string></i> Specifies the input transition time on the input pins. We recommend using a driver cell (<i>ext_driver_lib</i>, <i>ext_driver_cell</i>, <i>ext_driver_pin</i>) instead. If set the setting will be used together with the driving cell. This setting usually overrides the driver cell setting – see the documentation for your RTL synthesis tools for further details.
interconnect_mode	Type: <i>pick list</i>. (none, wireload, ple (RC Only)) Default: <i>none</i> RTL Tool: <i>All</i>

VAR Name	Description
	Wizard Name: <i>Interconnect Mode</i> Flow Run Switch: <code>-set_interconnect_mode <wireload none ple></code> Specifies whether to use wire load models. By default, DC uses 'none'. If no wire load models are specified in Catapult library the RTL synthesis tool may still select a wireload automatically. wireload: Enables the use of wireloads, if the wireload model, mode and/or selection group are specified those will be communicated to the RTL synthesis script through the generated synthesis script. none: In DC, this value will disable automatic wireload selection. No wireload settings will be written into the script even if specified.
iter_est_qmods	Type: <i>string</i>. Default: <code>{}</code>. RTL Tool: <i>DC</i> Wizard Name: <i>QMods for iterative fastest clock estimation</i> Flow Run Switch: <code>None</code> Specifies a space separated list of modules from the Catapult library that should use the iterative estimator to determine the area/delay of the fastest dataset for each qualified module (QMOD) of the specified module. eg. <code>mgc_add_8_0_4_0_9</code>. Typically, this list of modules includes the arithmetic operators in the library. eg. <code>{mgc_add mgc_csa mgc_csha mgc_div mgc_equal mgc_mod mgc_mul mgc_mul_pipe mgc_rem mgc_sqr}</code>
lef_library	This variable is obsolete. Use <code>libs_lef</code> instead.
libs_db	Type: <i>string</i>. Default: <code>""</code>. RTL Tool: <i>All</i> Wizard Name: <i>Synopsys DB Library Files</i> Flow Run Switch: <code>-libs_db <string></code> A space separated list of technology libraries in Synopsys db binary format. Catapult uses this value for running RTL synthesis using Design Compiler on the generated RTL netlist. Library Builder only uses this variable when you select "Design Compiler" as your RTL synthesis tool for characterization.
libs_lef	Type: <i>string</i>. Default: <code>""</code>. RTL Tool: <i>All</i> Wizard Name: <i>LEF Library files</i> Flow Run Switch: <code>-libs_lef <string></code> A space separated list of technology libraries in the LEF format. These files are not used by Design Compiler.
libs_liberty	Type: <i>string</i>. Default: <code>""</code>. RTL Tool: <i>All</i> Wizard Name: <i>Liberty Library files</i> Flow Run Switch: <code>-libs_liberty <string></code> A space separated list of leaf filenames of technology libraries in Liberty format. The path to these files must be included in the TechLibSearchPath option.
libs_map	Type: <i>string</i>. Default: <code>""</code>. RTL Tool: <i>Oasys</i> Wizard Name: <i>Oasys Layer Mapping file</i> Flow Run Switch: <code>-libs_map <string></code> The filename that contains the mappings between the layer names in the PTF and technology LEF files.
libs_powerpro	Type: <i>string</i>. Default: <code>""</code>. RTL Tool: <i>All</i> Wizard Name: <i>PowerPro Library files</i>

VAR Name	Description
	Flow Run Switch: <code>-libs_powerpro <string></code> A space separated list of leaf filenames of technology libraries in PowerPro (encrypted Liberty) or plain Liberty format. In Catapult's Low-Power option, this list overrides the value of the <code>libs_liberty</code> . This variable allows the Low Power option in Catapult to use different libraries for power analysis. The path to these files must be included in the TechLibSearchPath option.
<code>libs_ptf</code>	Type: <i>string</i> . Default: <code>""</code> . RTL Tool: <i>Oasys</i> Wizard Name: <i>Oasys PTF files</i> Flow Run Switch: <code>-libs_ptf <string></code> A space separated list of technology libraries in PTF format. This file contains the resistance and capacitance values per unit length for each layer. These files are not used by Design Compiler.
<code>link_library</code>	This variable is obsolete . Use <code>libs_db</code> and <code>libs_liberty</code> variables instead.
<code>libname</code>	Type: <i>string</i> Default: <code>""</code> . RTL Tool: <i>All</i> Wizard Name: <i>Library Name</i> Flow Run Switch: <code>-libname <string></code> The Library Wizard will append <code>"_ccs"</code> to the technology name in the liberty file. The Catapult library will be saved using this value as the root name with a <code>".lib"</code> suffix.
<code>libtitle</code>	Type: <i>string</i> Default: <code>""</code> . RTL Tool: <i>All</i> Wizard Name: <i>Library Title</i> Flow Run Switch: <code>-libtitle <string></code> In Catapult, this name will appear in the Compatible Libraries list in the Libraries Constraint Editor.
<code>no_boundary_optimization</code>	Type: <i>boolean</i> . Default: <i>true</i> . RTL Tool: <i>All</i> Wizard Name: <i>No Boundary Optimization</i> Flow Run Switch: <code>-set_no_boundary_optimization <bool></code> Disables boundary optimization during characterization if set to <i>true</i> . Catapult also uses the value set in the library when synthesizing the RTL netlist after the <i>extract</i> stage. You can override this value in Catapult for the Design Compiler flow .
<code>operating_conditions</code>	Type: <i>string</i> . Default: <code>""</code> . RTL Tool: <i>All</i> Wizard Name: <i>Operating Conditions</i> Flow Run Switch: <code>-set_operating_conditions <string></code> Specifies the name of the operating conditions specified in the technology library.
<code>operating_conditions_libs</code>	Type: <i>char</i> Default: <code>""</code> RTL Tool: <i>All</i> Wizard Name: <i>None</i> Flow Run Switch: <i>None</i> A list of <code><library name> <operating conditions name></code> pairs. Allows Catapult to be easily re-targeted to various operating conditions in the Low-Power flow. The path to these files must be included in the TechLibSearchPath option.
<code>set_driving_cell</code>	This variable is obsolete . Use <code>ext_driver_lib</code> , <code>ext_driver_cell</code> , and <code>ext_driver_pin</code> instead.
<code>set_load</code>	This variable used with DC is obsolete . Use <code>ext_load</code> instead.

VAR Name	Description
set_input_transition	This variable is obsolete . Use <i>input_transition</i> instead.
scan_registers	Type: <i>boolean</i> . Default: <i>false</i> . RTL Tool: <i>DC</i> Wizard Name: <i>Enable Scan Registers</i> Flow Run Switch: <code>-scan_registers <bool></code> Specifies whether to use regular flip-flops or scan registers in the RTL synthesis generated netlist.
significant_digits	This variable used with DC is obsolete . Use <i>fraction_digits_t</i> (for timing values) and <i>fraction_digits_c</i> (for capacitance values) instead.
synthetic_library	Type: <i>string</i> . Default: <i>{dw_foundation.sldb standard.sldb}</i> RTL Tool: <i>DC</i> Wizard Name: <i>Synthetic Libraries</i> Flow Run Switch: <code>-set_synthetic_library <string></code> Specifies a space separated list of DesignWare library files. <i>dw_foundation.sldb</i> <i>standard.sldb</i> is frequently used.
target_library	This variable is obsolete . Use <i>libs_db</i> , <i>libs_lef</i> , and <i>libs_liberty</i> variables instead.
technology	Type: <i>string</i> . Default: <i>TECHNOLOGY</i> RTL Tool: <i>All</i> Wizard Name: <i>Technology</i> Flow Run Switch: <code>-technology <string></code> Catapult libraries are associated with a vendor and technology names used for filtering. In Catapult, when these names are selected in the Libraries Constraint Editor, the library will appear in the Compatible Libraries list of the dialog box. If you are creating a library that must work with an existing Base Library, then the technology and vendor names must match. For example, vendor name "Sample" and technology name "065nm".
time_factor	Type: <i>float</i> . Default: <i>'1'</i> . RTL Tool: <i>All</i> Wizard Name: <i>Time Unit Factor</i> Flow Run Switch: <i>None</i> Must be equal to timing unit factor in technology library. Specifies the timing unit factor.
time_unit	Type: pick list (<i>ns</i> , <i>ps</i> , or <i>fs</i>). Default: <i>ns</i> RTL Tool: <i>All</i> Wizard Name: <i>Time Unit</i> Flow Run Switch: <code>-time_unit <ns ps fs></code> Must be equal to time unit in technology library. In conjunction with <i>time_factor</i> specifies the time unit.
vendor	Type: <i>string</i> . Default: <i>VENDOR</i> RTL Tool: <i>All</i> Wizard Name: <i>vendor</i> Flow Run Switch: <code>-vendor <string></code> Catapult libraries are associated with a vendor and technology names used for filtering. In Catapult, when these names are selected in the Libraries Constraint Editor, the library will appear in the Compatible Libraries list of the dialog box. If you are creating a library that must work with an existing Base Library, then the technology and vendor names must match. For example, vendor name "Sample" and technology name "065nm".
vth_constraint_percent	Type: <i>integer</i> . RTL Tool: <i>DC, Oasys</i> Wizard Name: <i>voltage threshold constraint percent</i>

VAR Name	Description
	Flow Run Switch: <code>-set_vth_constraint_percent <int></code> Specifies allowable percentage of low voltage threshold cells in the gate-level netlist
vth_constraint_type	Type: pick_list (none, hard, soft). Default: none RTL Tool: DC Wizard Name: voltage threshold marker type Flow Run Switch: <code>-set_vth_constraint_type <none hard soft></code> Specifies how rigidly the RTL synthesis tool will adhere to the VTH constraint percentage. By default, the percentage value is ignored (none).
vth_high_marker	Type: string. RTL Tool: DC, Oasys Wizard Name: High Vth threshold marker string Flow Run Switch: <code>-set_vth_high_marker <string></code> Specifies regex to identify high voltage threshold cells or libraries (depending on vth_marker_type value)
vth_low_marker	Type: string. RTL Tool: DC, Oasys Wizard Name: Low Vth threshold marker string Flow Run Switch: <code>-set_vth_low_marker <string></code> Specifies regex to identify low voltage threshold cells or libraries (depending on vth_marker_type value)
vth_marker_type	Type: pick_list (tech_lib, lib_cell). Default: tech_lib RTL Tool: DC Wizard Name: voltage threshold marker type Flow Run Switch: <code>-set_vth_marker_type <tech_lib lib_cell></code> Specifies whether the high/low voltage threshold cells should be identified by the library name or the cell name.
wait_for_design_license	Type: string. Default: {} RTL Tool: DC Wizard Name: Design License to wait for Flow Run Switch: <code>-set_wait_for_design_license <string></code> Specifies a list of DesignWare licenses that the DC tool should wait for if they are temporarily unavailable. The list items are space separated. The value of this variable is assigned to the DC variable <code>synlib_wait_for_design_license</code> in the generated DC script. This value will be the default setting for Catapult library users.
wire_load_libs	Type: string. Default: "" RTL Tool: All Wizard Name: None Flow Run Switch: None A list of <library name> <wireload name> pairs. Allows Catapult to easily select proper wireload based on the library name.
wire_load_mode	Type: string (top, enclosed, or segmented). Default: "top". RTL Tool: All Wizard Name: Wire Load Mode Flow Run Switch: <code>-set_wire_load_mode <top enclosed segmented></code> Corresponds to <code>set_wire_load_mode</code> in SDC. This library variable can be overridden by the <code>wire_load_mode</code> option in the Low-Power flow Options.
wire_load_model	Type: string. Default: "" RTL Tool: All Wizard Name: Wire Load Model Flow Run Switch: <code>-set_wire_load_model <string></code> Corresponds to <code>set_wire_load_model</code> in SDC. Set the <i>wireload model name</i>

VAR Name	Description
	only (no library name etc). This library variable can be overridden by the <i>wire_load_model</i> option in the Low-Power flow Options.
wire_load_selection_group	Type: <i>string</i> . Default: <i>""</i> . RTL Tool: <i>DC</i> Wizard Name: <i>Wire Load Selection Group</i> Flow Run Switch: <i>-set_wire_load_selection_group <string></i> Corresponds to <i>set_wire_load_selection_group</i> in <i>SDC</i> .

3.3. Editing Module Parameters

Module parameters typically correspond to the generics/parameters on the RTL.

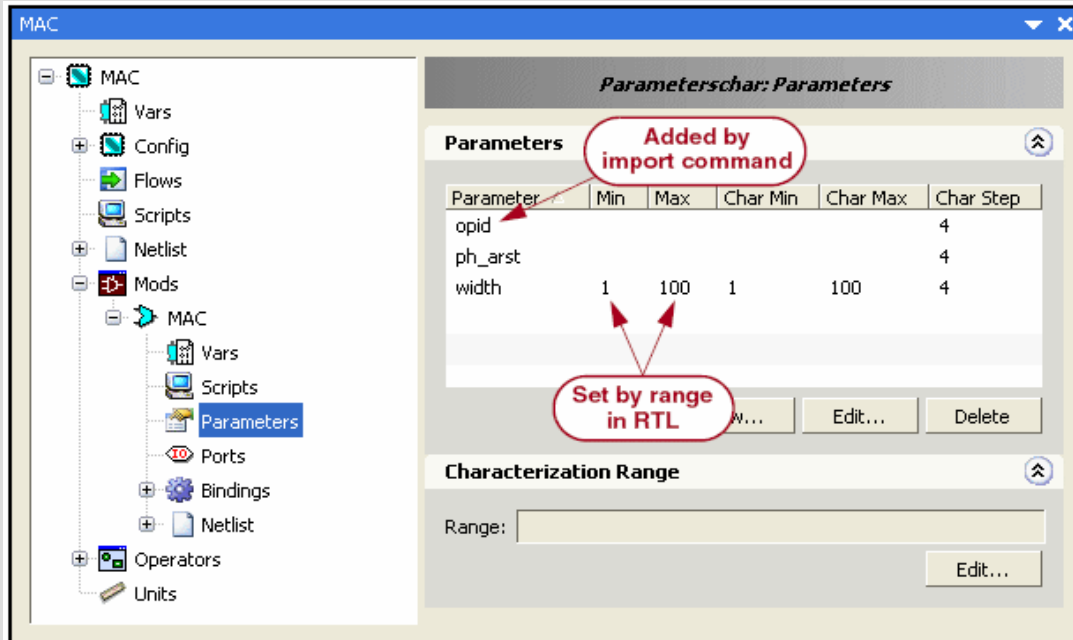
Note that for a base library the parameters of each module are set in the template and should not be edited.

The import command will add an additional parameter for interfaces, memories, and operators with state. For memories, a parameter called *ram_id* is added. For interfaces, a parameter called *rscid* is added. For custom operators with state, a parameter called *opid* is added. The user does not need to modify these parameters. For all other parameters the min/max range and min/max characterization range must be set. The min/max range tells Catapult the allowable range of the parameter. This controls things like how wide can a port be set on an interface, or how many words can a memory have. If the RTL generic/parameter has a range set on it, the import command will annotate this information for the min/max and characterization ranges. If the range is not specified in the RTL the user must set it manually. For example:

```
ENTITY MAC IS
GENERIC(width: natural range 1 to 100:= 2;
        ph_arst : natural := 1);
PORT (
    clk      : IN STD_LOGIC ;
    rst      : IN STD_LOGIC ;
    a : IN STD_LOGIC_VECTOR(width-1 DOWNT0 0) ;
    b : IN STD_LOGIC_VECTOR(width-1 DOWNT0 0) ;
```

```
c : OUT STD_LOGIC_VECTOR(width*2-1 DOWNT0 0)
) ;
END MAC ;
```

Illustration 29: Parameters Imported from Netlist



Parameters can be set manually by double-clicking on the parameter or selecting edit.

3.4. Editing Module Ports

Note that for a base library the ports for each module are set in the template and should not be edited.

Ports correspond to the ports on the RTL. Most of the port information is added during the RTL import, but there are some fields that must be set by the user.

- Input Register Setting**

Catapult requires that either the outputs of the core process (RTL) that it generates or the inputs of the interface/operator RTL IP are registered. If input register is set to true on the module port, then Catapult does not put a register on the corresponding output of the core process. This is shown in the following illustration.

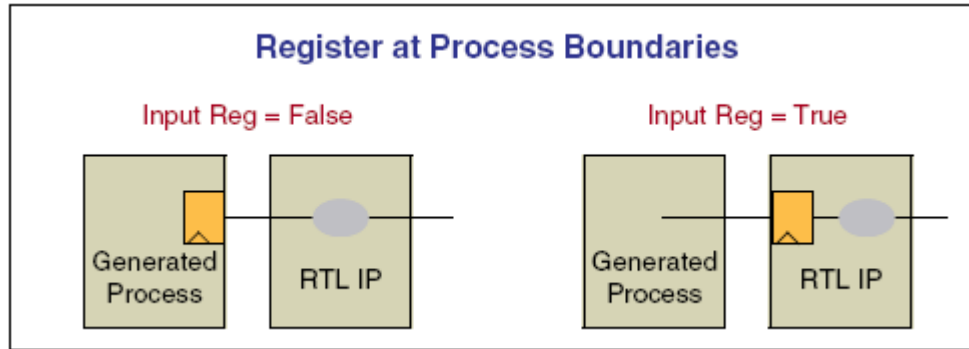


Illustration 30: Input Register Setting

There are cases where you may wish to set input register to false even though the interface/operator

RTL has input registers. If you require that the data and control inputs to the interface/operator RTL should be driven for multiple clock cycles, set the input register to false. This is only done for multi-cycle interface/operators.

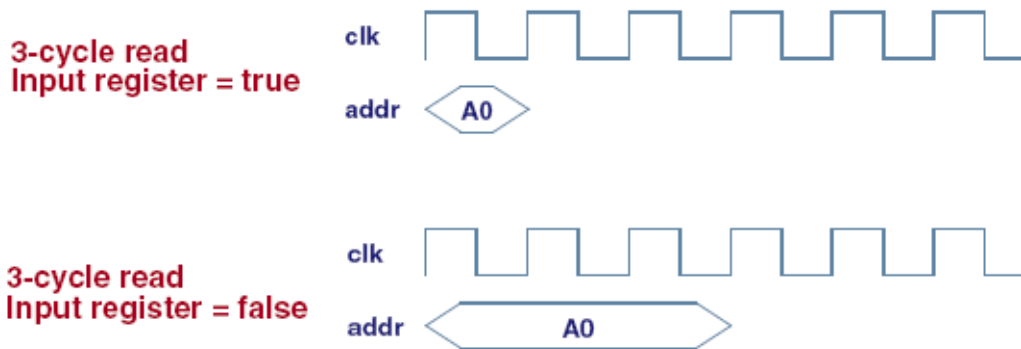


Illustration 31: Effects of Input Register Setting

- Sign Bit**

The port sign bit indicates that the port is signed or unsigned (1 == signed, 0 == unsigned). Catapult will sign extend inputs and outputs of signed ports. Single bit ports should leave the sign bit unassigned. Setting the sign bit on a single bit port results in `std_logic_vector(0 downto 0)`.

- Default Value**

The default value port setting indicates what value should be driven on an interface/operator input port when the interface/operator is not being read/written. This setting is used for ports like memory write enables, resets, and so on.

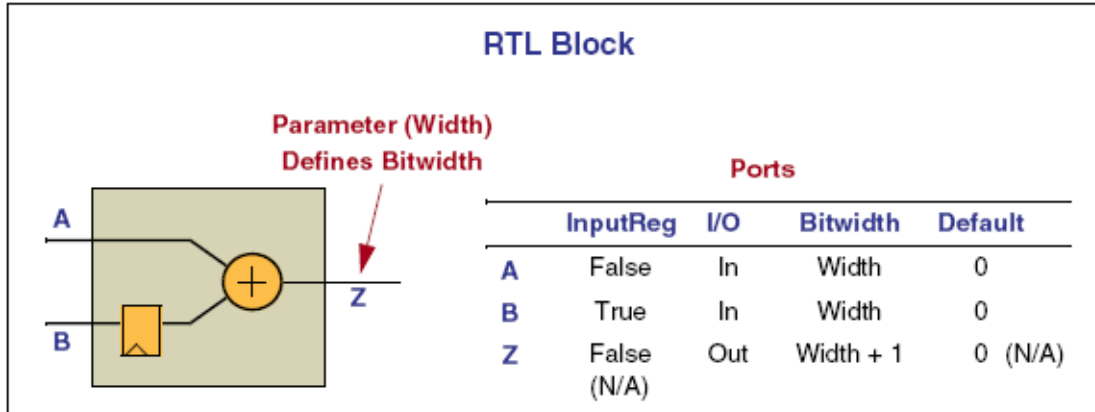


Illustration 32: Port Settings Overview

3.5. Editing Module Bindings

Bindings tell Catapult how to connect to the RTL interface/operator. They also allow timing and area information to be specified. There are usually at least two bindings for every interface/operator, the “All” binding, and the operator binding.

Operator bindings typically consist of read_port/write_port bindings for interfaces, read_ram/write_ram bindings for memories, built-in operators (add, mul, etc.), and user-defined for custom operators.

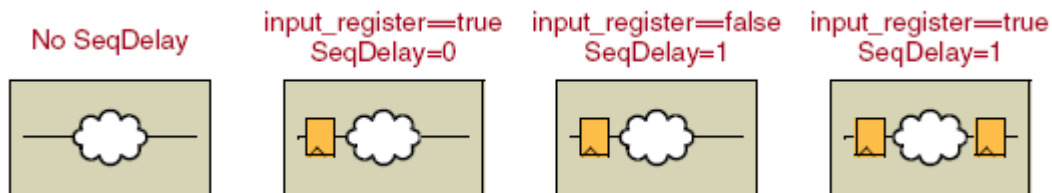


Illustration 33: Effect of Input Register on SeqDelay

All binding is typically used to set area and timing information. This is done via a property mapping. Property mappings allow module parameters to be set, or to map operator parameters to module parameters. The value on the properties on the binding can use any of the PARAMETERS as variables (eg. RAM 'area' may be determined by number of words and the bitwidth control).

The user selectable property can be set to true (1=true, 0=false) on individual parameters to make them editable by designers in the Catapult Constraint Editor. In a Catapult session, when editing the architectural constraints of a resource, any user selectable parameters of the library component is editable in the Resource Options field of the Constraint Editor. Each parameter field has a drop-down list of valid values to choose from.

Note that for a base library the bindings for each module are set in the template and should not be edited. The only exception is the property templates under the property mapping. Editing the property templates affects the interpolation quality and interpolation error. You will notice that every time you change the interpolation template the corresponding property is re-interpolated.

You can specify any properties you would like but the following properties are recognized by Catapult:

- **Area:** Total area of the module.
- **AreaSeq:** Sequential area of the module.
- **Delay:** Combinatorial delay, or clock-to-out time of sequential components.
- **InputDelay:** Input-to-clock delay of sequential components (excluding setup time).
- **InputSetup:** Setup time of the first row of registers for sequential components.
- **R2RDelay:** Clock-to-clock delay of the registers for sequential components (excluding setup time).
- **R2RSetup:** Setup time of the registers for sequential components except for the first row specified by InputSetup.
- **MinClkPrd:** Minimum clock period is computed as $\max(\text{InputSetup} + \text{InputDelay}, \text{R2RSetup} + \text{R2RDelay}, \text{Delay})$.
- **SeqDelay:** Sequential delay. Indicates the number of clock cycles required to complete the operation. If the ports associated with that operation have input registers, this property is needed. If the ports are asynchronous, this property is not needed.

For example:

- $\text{SeqDelay} = \text{Number of Component registers in any IO path} - 1$
- For components with `input_reg == true`, $\text{SeqDelay} = 0$. That means that Data available after first clock edge.
- No sequential delay property means the component is combinatorial.
- **InitDelay:** Number of cycles between accesses to this port. Default is $\text{SeqDelay} + 1$, but it can be set smaller for pipelined operation. For example, $\text{InitDelay} = 2$ means that the component can be pipelined down to $II=2$, but not $II=1$.
- **Always:** Must evaluate to true for all valid parameter configurations (QMODS). Used to limit the range of valid QMODS.

Property names are case sensitive. They can be set from the command line (example below) or from the GUI as shown in the following illustration.

```
library add \
/LIBS/<library_name>/MODS/<module_name>/BINDINGS/all/PROPERTY_MAPPING \
-- -Area width*20 -Delay 1.2 -SeqDelay 1 -InitDelay 1
```

The following illustration shows how to set the property in the GUI:

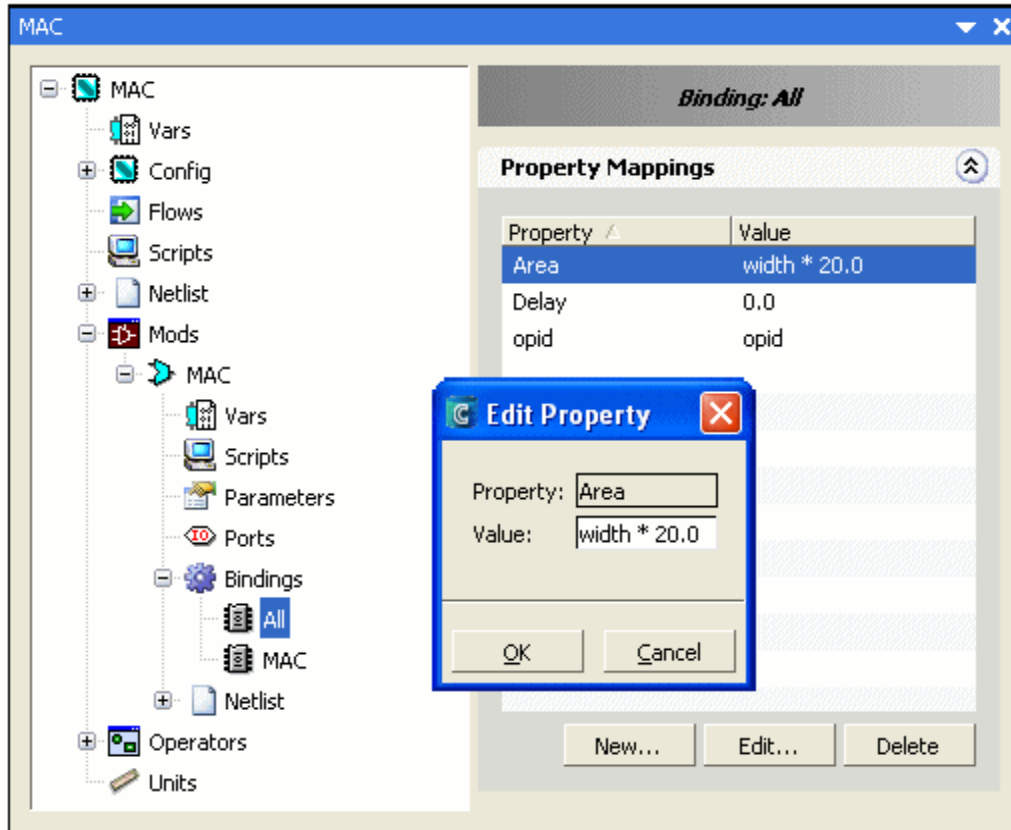


Illustration 34: Setting Property on the ALL Binding

3.6. Editing Module Pin Associations

Note that for a base library the pin associations for each module are set in the template and should not be edited.

Pin associations are used to connect operator ports to RTL ports and to tell Catapult how to bind the RTL component when netlisting the final VHDL or Verilog output. The pin associations are set on the operator binding. The following table describes the pin association types.

Table 8: Pin Associations

Pin	Description
Unbound	Left unconnected
OPERATOR_PIN	Binds to port on operator
CLOCK	Clock from Catapult process.
ENABLE	Clock enable from Catapult process.
S_RST	Synchronous reset from Catapult process
A_RST	Asynchronous reset from Catapult process.

EXTERNAL]	Connects the port of a bound internal or interface resource to an external port. Interface resources with EXTERNAL bindings must be bound.
DIRECT	Connects the port of a bound internal resource to an external port. DIRECT bindings are removed for interface resources.
GLOBAL	Connects the port of bound internal resources to a single external port when the internal port names are the same. GLOBAL bindings are removed for interface resources.
CONSTANT	Drives a constant value to the component port.
WAITON	Used for handshaking. The Catapult process will wait for the component port to be driven high.

When the RTL netlist is imported, the appropriate binding is automatically added based on the module type setting. If the module type is set to userop the operator ports must be defined before the pin associations can be made. If the inport, outport, ram, or rom module type is set, the operator binding is added and you must specify the pin associations. The following table describes the pins on each operator.

Table 9: Operator Pin Bindings

Binding	Data Port	Address
read_port	D	n/a
write_port	D	n/a
read_ram	D	I
write_ram	D	I

3.7. Editing Module Property Mappings

Property mappings allow operator parameters to be mapped to Module parameters which in turn may correspond to generics on the RTL. This allows an operator to control things like RTL port width, number of words in a memory, and so on. The [library import](#) command automatically maps the necessary operator parameters to their respective module parameters (such as width, id, etc.). If you are creating a library component manually, and that component uses built-in operators such as read_port or write_port, you must manually map the operator parameters to the module parameter in the bindings. The built-in interface and memory operators have the following parameters.

Table 10: Property Mapping

Param	read_port	write_port	read_ram	write_ram	Custom operator	Comment
width	X	X	X	X		Set based on interface or memory width arch constraint
size			X	X		Set based on number of array elements

ramid			X	X		Set for each ram instance
id	X	X				Set for each interface instance
opid					X	Set for operators with state

Chapter 4: Third-Party Information

The following HTML page provides information on third-party software that may be included in the Catapult product, including any additional license terms. This document is located at the following location within the Catapult install tree:

```
$MGC_HOME/shared/pkgs/sif_docs/legal/Catapult_OSS.html
```